

FORMATION EMBARQUÉE

TP — tp1 arduino station meteo

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 1 — Station météo Arduino/ESP32 (≈ 12 h, en 4 séances)

Objectif pédagogique : construire un système embarqué complet et non bloquant — capteur, affichage, journalisation, réseau — en appliquant `millis()`, la FSM, I2C, SPI et MQTT. C'est le projet vitrine du début de portfolio.

 **Version détaillée** : chaque séance existe en fiche minutée avec code complet, erreurs fréquentes et travail à la maison — [Séance 1](#) · [Séance 2](#) · [Séance 3](#) · [Séance 4](#). Cette page reste la vue d'ensemble et la grille de notation.

Matériel / alternative simulateur

- Arduino Uno ou Nano (séances 1-3), ESP32 (séance 4)
- DHT22, écran OLED SSD1306 (I2C), potentiomètre, LED rouge, buzzer (option), module carte SD (option séance 3)
- **Sans matériel** : tout le TP (séance 4 comprise) se fait sur <https://wokwi.com> — crée un projet « Arduino Uno » puis « ESP32 ».

Bibliothèques : `DHT sensor library` (Adafruit), `Adafruit SSD1306`, `Adafruit GFX`, et séance 4 : `PubSubClient`.

Règle du TP : un seul `delay()` toléré dans tout le projet (celui du `setup()` éventuel). Chaque séance se termine par un commit Git.

Séance 1 (2 h) — Socle non bloquant

Étape 1.1 — Câblage

DHT22 : VCC → 5V, GND → GND, DATA → D2 (avec pull-up 10 kΩ si module nu). LED rouge : D8 → résistance 220 Ω → LED → GND. Potentiomètre : extrémités 5V/GND, curseur → A0.

Étape 1.2 — Squelette à tâches périodiques

Écris le squelette suivant et **vérifie au moniteur série que les deux périodes tiennent indépendamment** :

```

#include <DHT.h>
DHT dht(2, DHT22);

uint32_t t_mesure = 0, t_affiche = 0;
float temperature = NAN, humidite = NAN;

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(8, OUTPUT);
}

void loop() {
  uint32_t now = millis();

  if (now - t_mesure >= 2000) {          // tâche 1 : mesurer (2 s)
    t_mesure = now;
    float t = dht.readTemperature(), h = dht.readHumidity();
    if (!isnan(t)) temperature = t;
    if (!isnan(h)) humidite = h;
  }

  if (now - t_affiche >= 1000) {         // tâche 2 : afficher (1 s)
    t_affiche = now;
    Serial.print("T="); Serial.print(temperature);
    Serial.print("C H="); Serial.print(humidite); Serial.println("%");
  }
}

```

✓ **Point de contrôle 1** : débranche la broche DATA du DHT en cours de fonctionnement. Le programme doit continuer d'afficher (les dernières valeurs valides), pas se figer ni afficher `nan` en boucle. Si ce n'est pas le cas, revois la gestion de `isnan`.

Étape 1.3 — Seuil et hystérésis

Ajoute : consigne = potentiomètre (10-40 °C), LED rouge allumée si `temperature > consigne + 0,5` et éteinte si `< consigne - 0,5` (reprennds le TD 03 exercice 3). ✓ **Point de contrôle 2** : en soufflant sur le capteur, la LED ne « bat » pas autour du seuil.

Séance 2 (3 h) — Affichage OLED et architecture

Étape 2.1 — OLED I2C

SDA → A4, SCL → A5 (Uno). Adresse 0x3C. Affiche T/H en gros, consigne en petit, un pictogramme d'alerte si dépassement. Rafraîchis l'écran **au plus 5 fois/seconde** (tâche périodique dédiée) — jamais à chaque tour.

Étape 2.2 — Refactorisation en modules

Impose-toi la structure de fichiers suivante (onglets de l'IDE ou fichiers `.h/.cpp`) :

```
station_meteo/  
├─ station_meteo.ino      // setup/loop : UNIQUEMENT l'orchestration  
├─ capteur.h / .cpp       // lecture DHT + validité + dernier relevé  
├─ affichage.h / .cpp     // tout l'OLED (personne d'autre n'inclut Adafruit_SSD1306)  
└─ alerte.h / .cpp        // hystérésis + LED (+ buzzer)
```

✓ **Point de contrôle 3** : le `.ino` fait moins de 60 lignes et ne contient aucun appel direct aux bibliothèques Adafruit/DHT. Critère d'architecture : « pour changer d'écran, je ne touche qu'à `affichage.cpp` ».

Séance 3 (3 h) — FSM d'interface et journalisation

Étape 3.1 — Bouton et machine d'états d'affichage

Ajoute un bouton (D3, `INPUT_PULLUP`, anti-rebond du TD 03). Chaque appui change de page : `PAGE_MESURES → PAGE_MINMAX → PAGE_CONSIGNE → ...`. Implémente en `enum class` + `switch` (FSM du module 01). La page MIN/MAX affiche les extrêmes depuis le démarrage, remis à zéro par appui long (> 2 s — à détecter avec `millis()`, évidemment).

Étape 3.2 — (option matériel) Carte SD

Module SD en SPI (CS=D10). Toutes les 60 s, ajoute une ligne CSV : `millis;temperature;humidite`. Gère l'absence/retrait de carte sans planter (l'écriture échoue → on continue, on réessaie plus tard, on le signale sur la page d'accueil).

✓ **Point de contrôle 4** : appui court = page suivante, appui long = RAZ min/max, et le tout reste fluide pendant les mesures et l'écriture SD.

Séance 4 (4 h) — Version connectée ESP32 + MQTT

Étape 4.1 — Portage

Ouvre un projet ESP32 (Wokwi : carte « ESP32 DevKit »). Adapte : DHT sur GPIO 4, OLED sur 21/22 (I2C par défaut), LED sur GPIO 2. Tes modules `capteur/affichage/alerte` doivent passer **sans modification** — c'est le test de la séance 2.

Étape 4.2 — Wi-Fi + MQTT

```
#include <WiFi.h>
#include <PubSubClient.h>

WiFiClient net;
PubSubClient mqtt(net);

void connecter() {
    WiFi.begin("Wokwi-GUEST", ""); // réseau simulé de Wokwi
    while (WiFi.status() != WL_CONNECTED) delay(250); // seul delay toléré (init)
    mqtt.setServer("broker.hivemq.com", 1883); // broker public de test
    mqtt.connect("station-TONPRENOM"); // id UNIQUE, sinon déconnexions
}

void publier(float t, float h) {
    char json[64];
    snprintf(json, sizeof json, "{\"t\":%.1f,\"h\":%.1f}", t, h);
    mqtt.publish("formation/TONPRENOM/meteo", json);
}
```

Publie toutes les 10 s (tâche périodique). Appelle `mqtt.loop()` à chaque tour de `loop()`. Gère la reconnexion : si `!mqtt.connected()`, retente **au plus une fois toutes les 5 s** (pas en rafale).

Étape 4.3 — Vérification de bout en bout

Sur ton PC : `mosquitto_sub -h broker.hivemq.com -t "formation/TONPRENOM/#" -v` (ou l'outil web [hivemq.com/demos/websocket-client](https://broker.hivemq.com/demos/websocket-client)).  **Point de contrôle 5** : les JSON arrivent, et coupent proprement quand tu « débranches » le Wi-Fi dans la simulation, puis reprennent seuls.

Grille d'auto-évaluation du TP (à remplir honnêtement)

Critère	Points
Aucun <code>delay()</code> hors init ; périodes respectées	/4
Panne capteur, absence SD, perte Wi-Fi : gérées sans blocage	/4
Architecture en modules, <code>.ino</code> d'orchestration seul	/4
FSM d'affichage propre (enum + switch, transitions nettes)	/3
MQTT fonctionnel avec reconnexion raisonnée	/3
Git : ≥ 4 commits datés, messages clairs, README avec photo/schéma	/2
Total	/20

Pour aller plus loin : OTA (mise à jour par Wi-Fi), sommeil profond de l'ESP32 entre deux mesures (autonomie sur batterie), Node-RED côté PC pour tracer les courbes — chacun de ces ajouts est un futur commit de portfolio.

FORMATION EMBARQUÉE

TP — tp2 vhdI uart

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 2 — Chaîne UART complète en VHDL (≈ 8 h, en 3 séances)

Objectif pédagogique : concevoir, simuler et valider un périphérique matériel complet — émetteur + récepteur UART avec anti-métastabilité et sur-échantillonnage — en appliquant la méthode professionnelle : spécification → schéma → code → testbench → chronogramme.

 **Fiches minutées séance par séance** : [Séance 1](#) · [Séance 2](#) · [Séance 3](#). Cette page est la vue d'ensemble et la grille de notation.

Outils

- **GHDL + GTKWave** (gratuits) : `sudo apt install ghdl gtkwave` ou <https://www.edaplayground.com> (rien à installer).
- Aucune carte FPGA nécessaire ; une section finale optionnelle donne les contraintes pour Basys 3 / Tang Nano.

Script de travail (à créer une fois, `simule.sh`) :

```
#!/bin/sh
set -e
ghdl -a --std=08 "$@"          # analyse des fichiers passés en argument
TB=$(basename "${!#}" .vhd)    # dernier fichier = testbench
ghdl -e --std=08 "$TB"
ghdl -r --std=08 "$TB" --wave="$TB.ghw" --stop-time=5ms
gtkwave "$TB.ghw" &
```

Séance 1 (2 h) — Spécification et émetteur

Étape 1.1 — Écrire la spécification AVANT le code

Recopie et complète ce tableau (c'est un livrable) :

Paramètre	Valeur	Justification
Format	8N1	standard
Baudrate	115 200	debug usuel
F horloge	50 MHz	générique
Ticks par bit	$50e6/115200 = 434$ (arrondi)	erreur = ? % (à calculer)
Erreur cumulée sur 10 bits	?	doit rester $\ll \frac{1}{2}$ bit

Calcul attendu : 434 ticks → baud réel 115 207, erreur ≈ 0,006 %, soit 0,06 % sur la trame :

négligeable (la limite communément admise est $\sim 2\%$). **Savoir faire ce calcul est un objectif du TP.**

Étape 1.2 — Émetteur

Reprends `uart_tx` du TD 04 exercice 5 (ou réécris-le de mémoire, c'est mieux). Simule avec son testbench.  **Point de contrôle 1** : au chronogramme, mesure avec les curseurs de GTKWave la durée d'un bit — elle doit valoir $434 \text{ cycles} \pm 0$.

Séance 2 (3 h) — Le récepteur (le morceau noble)

Étape 2.1 — Comprendre le sur-échantillonnage

Le récepteur n'a pas l'horloge de l'émetteur : il se resynchronise sur le front descendant du start, puis échantillonne chaque bit **en son milieu** :

```
ligne RX : ───┐ start ┌─┐ b0 ┌─┐ b1 ...
              └──┘      └─┘
              ▲        ▲        ▲
              ½ bit   1 bit  1 bit   ← instants d'échantillonnage
```

Détection du start à mi-bit : si la ligne est encore à 0, c'est un vrai start (sinon, c'était un parasite → retour au repos). C'est un **filtre anti-glitch gratuit** — note-le dans ton compte rendu.

Étape 2.2 — Coder `uart_rx`

Squelette imposé (complète les `-- À FAIRE`) :

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity uart_rx is
    generic ( F_CLK : natural := 50_000_000; BAUD : natural := 115_200 );
    port (
        clk      : in  std_logic;
        rx       : in  std_logic;                      -- ligne série (ASYNCHRONE)
        donnee   : out std_logic_vector(7 downto 0);
        valide   : out std_logic                      -- '1' pendant 1 cycle
    );
end entity;

architecture rtl of uart_rx is
    constant TPB : natural := F_CLK / BAUD;
    type t_etat is (REPOS, VERIF_START, DONNEES, STOP);
    signal etat : t_etat := REPOS;
    signal sync0, sync1 : std_logic := '1';           -- synchroniseur 2 FF
    signal cpt : natural range 0 to TPB - 1 := 0;
    signal idx : natural range 0 to 7 := 0;
    signal shreg : std_logic_vector(7 downto 0);
begin
    process (clk)
    begin
        if rising_edge(clk) then
            sync0 <= rx;                                -- JAMAIS rx directement
            sync1 <= sync0;
            valide <= '0';                               -- impulsion par défaut

            case etat is
                when REPOS =>
                    if sync1 = '0' then                  -- front de start ?
                        cpt <= 0; etat <= VERIF_START;
                    end if;
                when VERIF_START =>
                    if cpt = TPB/2 - 1 then              -- milieu du start
                        if sync1 = '0' then              -- toujours bas : vrai start
                            cpt <= 0; idx <= 0; etat <= DONNEES;
                        else
                            etat <= REPOS;               -- parasite : on abandonne
                        end if;
                    else
                        cpt <= cpt + 1;
                    end if;
                when DONNEES =>
                    -- À FAIRE : attendre TPB cycles, échantillonner sync1
                    -- dans shreg(idx) (LSB d'abord), 8 fois
                when STOP =>
                    -- À FAIRE : attendre TPB cycles ; si sync1='1',
                    -- donnee <= shreg et valide <= '1' ; retour REPOS
            end case;
        end if;
    end process;
end rtl;

```

```
end process;  
end architecture;
```

Étape 2.3 — Testbench en boucle (le juge de paix)

Le test le plus élégant : **brancher ton TX sur ton RX** et vérifier que ce qui sort est ce qui est entré — pour les 256 valeurs possibles :

```
architecture sim of tb_boucle is  
    signal clk, tx_start, fil, busy, valide : std_logic := '0';  
    signal d_in, d_out : std_logic_vector(7 downto 0);  
begin  
    emetteur : entity work.uart_tx port map (clk, tx_start, d_in, fil, busy);  
    recepteur : entity work.uart_rx port map (clk, fil, d_out, valide);  
    clk <= not clk after 10 ns; -- 50 MHz  
  
    stim : process  
    begin  
        for i in 0 to 255 loop -- test EXHAUSTIF  
            d_in <= std_logic_vector(to_unsigned(i, 8));  
            wait until rising_edge(clk);  
            tx_start <= '1'; wait until rising_edge(clk); tx_start <= '0';  
            wait until valide = '1';  
            assert d_out = d_in  
                report "echec pour " & integer'image(i) severity failure;  
            wait until busy = '0';  
        end loop;  
        report "BOUCLE OK : 256/256 octets";  
        wait;  
    end process;  
end architecture;
```

✓ **Point de contrôle 2** : **BOUCLE OK : 256/256** . Si un octet échoue, ouvre le chronogramme à cet octet précis et compare les instants d'échantillonnage — 90 % des bugs sont un décompte décalé d'un cycle (erreur « off by one » sur **cpt = TPB - 1** vs **TPB**).

Séance 3 (3 h) — Robustesse et intégration

Étape 3.1 — Tests de robustesse (à ajouter au testbench)

1. **Glitch** : force **fil <= '0'** pendant 1 µs (< ½ bit) hors trame ; **valide** ne doit **jamais** se lever.
2. **Horloges désaccordées** : instancie l'émetteur avec **F_CLK => 50_000_000** et le récepteur avec **F_CLK => 49_000_000** (−2 %) : la boucle des 256 doit encore passer. À −5 %, observe et explique l'échec (l'erreur cumulée dépasse ½ bit au 8^e bit).
3. **Trame sans stop** (émetteur trafiqué ou stimulus manuel) : le récepteur doit revenir au repos sans se verrouiller.

Étape 3.2 — Module d'écho

Assemble un `top_echo` : tout octet reçu est réémis (RX.valide → TX.tx_start, avec gestion du cas « TX occupé » : un registre d'attente d'un octet suffit — documente ce choix). Testbench : envoie « VHDL », vérifie l'écho.

Étape 3.3 — (option carte réelle)

Basys 3 : `create_clock -period 10.0` sur l'horloge 100 MHz (adapte le generic !), broches USB-UART B18 (RX) / A18 (TX) dans le `.xdc` ; ouvre un terminal série à 115 200 : ce que tu tapes revient à l'écran — ton matériel parle au PC.

Livrables et barème

Livrable	Points
Spécification remplie avec calcul d'erreur de baudrate	/3
<code>uart_rx</code> complet : synchroniseur, vérif à mi-start, milieu de bit	/5
Boucle 256/256 verte	/4
Les 3 tests de robustesse + explication du -5 %	/4
Écho fonctionnel avec cas « TX occupé » traité et documenté	/3
Compte rendu : chronogrammes annotés (captures GTKWave)	/1
Total	/20

Ce que ce TP t'a fait pratiquer : FSM matérielles, synchronisation inter-domaines, sur-échantillonnage, testbench exhaustif et tests aux limites — le quotidien exact d'un ingénieur FPGA.

FORMATION EMBARQUÉE

TP — tp3 siemens remplissage

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 3 — Station de remplissage sous TIA Portal + PLCSIM (≈ 10 h, en 4 séances)

Objectif pédagogique : mener un mini-projet d'automatisme **dans les règles de l'art** : analyse fonctionnelle → liste d'E/S → GRAFCET → blocs structurés → HMI → recette de test. C'est la démarche qu'on te demandera en bureau d'études, pas seulement « faire marcher ».

 **Fiches minutées séance par séance** : [Séance 1 — Analyse](#) · [Séance 2 — Programme](#) · [Séance 3 — PLCSIM](#) · [Séance 4 — HMI](#).

Outils

TIA Portal (essai 21 jours ou licence école) avec **PLCSIM** et **WinCC Basic**. CPU projet : S7-1214C DC/DC/DC (ou n'importe quelle 1200/1500 — PLCSIM simule tout).

Cahier des charges (le client dit :)

Une ligne remplit des bouteilles :

1. Un **convoyeur** amène les bouteilles. Une **cellule** détecte la bouteille sous le bec.
2. Bouteille en place → convoyeur stoppé, **électrovanne** ouverte pendant le **temps de remplissage réglable** (2 à 10 s, réglé depuis l'écran).
3. Vanne fermée → 1 s de stabilisation → le convoyeur repart.
4. Un **compteur de lot** : l'opérateur règle la taille du lot (ex. 12) ; lot atteint → la ligne s'arrête, message « lot terminé », repart après acquittement.
5. Modes **AUTO / MANU** (en MANU : commandes directes convoyeur et vanne, avec les sécurités actives).
6. Défauts : **bourrage** (cellule occupée > 15 s hors remplissage), **arrêt d'urgence** (NF câblé). Tout défaut → arrêts sûrs + voyant + alarme HMI + acquittement obligatoire.

Séance 1 (2 h) — Analyse (aucune ligne de code)

Livrable 1 : liste d'E/S

À produire en tableau — en voici le début, complète-le :

Repère	Description	Type	Adresse	Actif
S1	BP Marche	DI	%I0.0	NO
S2	BP Arrêt	DI	%I0.1	NO
S3	Arrêt d'urgence	DI	%I0.2	NF
S4	Cellule bouteille présente	DI	%I0.3	NO
S5	Sélecteur AUTO/MANU	DI	%I0.4	—
S6	BP Acquit défauts	DI	%I0.5	NO
KM1	Convoyeur	DO	%Q0.0	—
YV1	Électrovanne remplissage	DO	%Q0.1	—
H1	Voyant défaut	DO	%Q0.2	—

Livrable 2 : GRAFCET de production (papier/dessin)

Trace le GRAFCET point de vue partie commande. Attendu :

```

0 Attente      (convoyeur arrêté)
  + marche ET auto ET aucun_defaut
10 Convoyage   N: KM1
  + S4 (bouteille détectée)
20 Remplissage N: YV1 ; tempo T_remplissage (réglable)
  + tempo écoulée
30 Stabilisation ; tempo 1 s ; P1: compteur := compteur + 1
  + tempo écoulée ET compteur < taille_lot → retour 10
  + tempo écoulée ET compteur >= taille_lot → 40
40 Lot terminé (tout arrêté, message)
  + acquit → remise compteur à 0 → 0

```

Livrable 3 : analyse des défauts

Pour **chaque** défaut, un tableau : détection, action immédiate, condition de réarmement. (Bourrage : S4 ET NOT étape 20/30 maintenu 15 s ; action : KM1 et YV1 coupés ; réarmement : disparition cause + S6.)

✅ **Point de contrôle 1** : fais relire ton GRAFCET à quelqu'un (ou relis-le à 24 h d'écart) : chaque transition doit être une condition *observable par un capteur* — « la bouteille est pleine » n'est pas un capteur, « la tempo est écoulée » oui.

Séance 2 (3 h) — Programme structuré

Crée le projet, la CPU, la table de variables (celle du livrable 1 + %M /DB). Structure **imposée** des blocs :

Bloc	Type	Rôle
Main [OB1]	OB	appelle les FC/FB dans l'ordre, RIEN d'autre
FC_Modes	FC	calcule <code>auto_actif</code> , <code>manu_actif</code> , demande marche/arrêt (auto-maintien)
FB_Defaults	FB	AU, bourrage (TON 15 s), mémorisation, acquit → <code>aucun_defaut</code>
FB_Sequence	FB	le GRAFCET en SCL (<code>CASE #etape OF</code>)
FC_Sorties	FC	seul bloc qui écrit %Q : combine séquence + MANU + défauts
DB_IHM	DB global	tout ce que voit/écrit l'écran : consignes, états, compteur

Points imposés (et pourquoi) :

1. **Un seul bloc écrit les sorties** (`FC_Sorties`). Quand un technicien cherchera « pourquoi KM1 ne tourne pas », il ouvrira UN réseau. Sorties éparpillées = projet inmaintenable (et double affectation = bug sournois : dernière écriture gagne).
2. La coupure de sécurité est **en dernier**, dans `FC_Sorties` : `KM1 := (cmd_auto OR cmd_manu) AND aucun_defaut AND NOT au_declenche` — quelle que soit la fantaisie de la séquence, un défaut coupe.
3. La séquence en SCL suit le GRAFCET **étape pour étape**, avec les numéros du dessin en commentaire. Le dessin est le document de référence ; le code n'est que sa traduction.

Extrait attendu de `FB_Sequence` (à compléter) :

```

CASE #etape OF
  0: // Attente
    IF "DB_IHM".marche_demandee AND #auto AND #aucun_defaut THEN
      #etape := 10;
    END_IF;
  10: // Convoyage
    IF "S4_bouteille" THEN #etape := 20; END_IF;
  20: // Remplissage
    #t_rempli(IN := TRUE,
              PT := DINT_TO_TIME("DB_IHM".temps_remplissage_ms));
    IF #t_rempli.Q THEN
      #t_rempli(IN := FALSE);
      #etape := 30;
    END_IF;
    // ... 30 et 40 : à toi ...
END_CASE;

```

✓ **Point de contrôle 2** : compilation sans erreur ni avertissement, et une **table de visualisation**

`Visu_Cycle` préparée avec : étape, %I, %Q, compteur, tempo.

Séance 3 (2 h) — Mise au point sous PLCSIM

Recette de test **écrite avant de tester** (livrable 4) — modèle :

#	Scénario	Actions (forçages)	Résultat attendu	OK ?
1	Cycle nominal	marche, puis S4 pulsé	10 → 20 → 30 → 10, compteur +1	
2	Lot de 3	taille_lot=3, 3 passages	étape 40, message	
3	AU en plein remplissage	S3 → 0 pendant étape 20	YV1 ET KM1 retombent < 1 cycle, défaut mémorisé	
4	Réarmement sans acquit	S3 → 1 seul	rien ne repart	
5	Bourrage	S4 maintenu 16 s hors remplissage	défaut bourrage	
6	MANU	sélecteur MANU, cmd vanne	vanne suit la commande, sécurités actives	
7	Consigne hors bornes	temps=99 s depuis la table	borné à 10 s (le programme borne !)	

Déroule-la dans PLCSIM en forçant les entrées depuis la table de visualisation. **Chaque écart = correction + re-déroulé complet** de la recette (une correction peut casser un scénario qui passait).

✅ **Point de contrôle 3** : les 7 scénarios verts, colonne OK datée.

Séance 4 (3 h) — HMI WinCC

Ajoute un pupitre KTP700 Basic. Trois vues :

- Conduite** : synoptique (convoyeur, bec, bouteille), boutons Marche/Arrêt (à impulsion ! cf. TD 07 ex. 4), état en clair (« Remplissage 3/12 »), compteur de lot, voyant mode.
- Réglages** : temps de remplissage (champ borné 2-10 s) et taille de lot (1-99) — protégés par le niveau utilisateur « Régleur » (mot de passe), car un opérateur ne modifie pas les recettes.
- Alarmes** : fenêtre des alarmes TOR (AU, bourrage, lot terminé en classe « avertissement »), bouton d'acquiescement.

Simule pupitre + PLCSIM ensemble et rejoue les scénarios 2, 3 et 5 **depuis l'écran**.

Livrables et barème

Livrable	Points
Liste d'E/S + GRAFCET + analyse des défauts (séance 1)	/4
Structure de blocs conforme, sorties centralisées, sécurité en aval	/5
Séquence SCL fidèle au GRAFCET, tempos et compteur corrects	/4
Recette de test écrite ET déroulée (7 scénarios)	/4
HMI 3 vues, boutons à impulsion, consignes bornées, alarmes	/3
Total	/20

Transfert de compétence : cette démarche (analyse → E/S → GRAFCET → structure → recette) est exactement celle du TP 4 côté Schneider — tu constateras que 80 % du travail est indépendant de la marque.

FORMATION EMBARQUÉE

TP — tp4 schneider cuve

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 4 — Gestion de cuve sous EcoStruxure + Modbus (≈ 8 h, en 3 séances)

Objectif pédagogique : refaire la démarche du TP 3 dans l'écosystème Schneider **et** ouvrir l'automate vers l'extérieur : table d'échange Modbus TCP lue par un client PC (Python ou ton programme Java du module 05). C'est le pont automatisme ↔ informatique.

 **Fiches minutées séance par séance** : [Séance 1 — Programme M221](#) · [Séance 2 — Table Modbus](#) · [Séance 3 — Supervision PC](#).

Outils

- **EcoStruxure Machine Expert – Basic** (gratuit) + son simulateur M221.
- Un client Modbus : `pip install pymodbus` (ou QModMaster en graphique).
- Prérequis : TP 3 terminé (la méthode n'est pas re-détaillée).

Cahier des charges

Une cuve alimente un procédé :

1. Mesure de **niveau analogique** 0-10 V sur `%IW0.0` (0..1000 en brut simulateur) représentant 0-100 %.
2. **Deux pompes de remplissage** P1/P2 : régulation à hystérésis — démarrage sous 40 %, arrêt au-dessus de 60 %. **Alternance** : à chaque cycle de remplissage, on utilise la pompe la moins utilisée (reprenons ton bloc du TD 08 exercice 2).
3. **Vanne de soutirage** commandée par le procédé aval (entrée TOR simulée).
4. Sécurités : **niveau très haut** (> 90 % : tout remplissage interdit, alarme), **niveau très bas** (< 5 % : soutirage interdit — protection de la pompe aval), **débordement capteur** (mesure figée > 30 s = capteur HS → défaut, positions sûres).
5. **Table d'échange Modbus TCP** pour la supervision (états, niveau, heures pompes, commandes, mot de vie).

Séance 1 (3 h) — Programme M221

Étape 1.1 — Projet et configuration

Nouveau projet Machine Expert Basic → TM221CE16R (version Ethernet). Onglet configuration : entrée analogique 0-10 V, adresse IP statique (192.168.1.50 pour la suite — le simulateur l'émule).

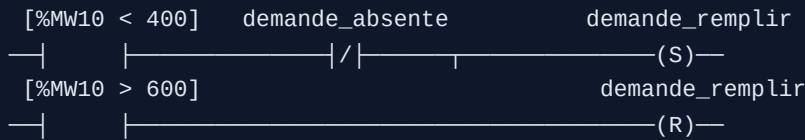
Étape 1.2 — Mise à l'échelle et symboles

- `%IW0.0` (0..1000) → `%MW10` = niveau en ‰ (0..1000 = 0..100,0 ‰) — garde les entiers ×10 (conventions du TD 08 ex. 3).

- Table de symboles complète AVANT de programmer (comme au TP 3) : `NIVEAU_POURMILLE (%MW10)` , `CMD_P1 (%Q0.0)` , `CMD_P2 (%Q0.1)` , `VANNE_SOUTIRAGE (%Q0.2)` , `DEF_CAPTEUR (%M20)` , etc.

Étape 1.3 — Les 4 fonctions, dans 4 sections nommées

- Régulation hystérésis (LADDER avec blocs comparaison) :



- Alternance des pompes** : transcris ton bloc du TD 08 (Machine Expert Basic n'a pas de FB utilisateur : implémente en LADDER + mots `%MW30/31` pour les compteurs d'heures, ou en liste d'opérations dans une section dédiée — documente le choix).
- Sécurités** — section « la plus en aval », seule à écrire les `%Q` : niveau > 900 ⇒ `CMD_P1 = CMD_P2 = 0` quoi qu'il arrive ; niveau < 50 ⇒ vanne fermée ; `DEF_CAPTEUR` ⇒ tout à l'état sûr (pompes coupées, vanne fermée).
- Chien de garde capteur** : si `%MW10` n'a pas varié de $\pm 2\%$ pendant 30 s **alors que** une pompe tourne ou la vanne est ouverte (sinon un niveau immobile est normal !) → `DEF_CAPTEUR` . Utilise un `%TM` 30 s remis à zéro à chaque variation.

✓ **Point de contrôle 1** : au simulateur, en pilotant `%IW0.0` à la souris : hystérésis correcte (départ sous 40 %, arrêt à 60 %), alternance visible d'un cycle au suivant, sécurité niveau haut prioritaire sur tout.

Séance 2 (2 h) — Table d'échange Modbus

Étape 2.1 — Document de table (livrable clef)

Reprends le format du TD 08 exercice 3 et adapte :

Mot	Sens	Contenu	Codage
%MW100	API → PC	État : b0=P1, b1=P2, b2=vanne, b3=nivTH, b4=nivTB, b5=defCapteur	bits
%MW101	API → PC	Niveau	‰
%MW102/103	API → PC	Heures P1 / P2	h ×10
%MW104	API → PC	Mot de vie	+1/s
%MW110	PC → API	Commande : b0=autoriser remplissage, b1=acquit	bits
%MW111	PC → API	Consigne haute (arrêt pompes)	‰, borné 500..800

Sur M221, les `%MW` sont **nativement accessibles en Modbus TCP** (holding registers, adresse = numéro du mot) : il n'y a rien à « publier », mais il faut recopier explicitement états → `%MW100..104` et lire/borner

`%MW110/111` dans une section « Échange » du programme. **Jamais** le PC n'écrit directement une `%Q` : il écrit une *demande* que l'automate valide.

Étape 2.2 — Mot de vie et bornage

- `%MW104` : +1 chaque seconde (bit système `%S6` + front + bloc opération).
- `%MW111` bornée à chaque cycle : `IF < 500 THEN 500 ; IF > 800 THEN 800` — l'automate ne fait **jamais** confiance au réseau.

Séance 3 (3 h) — Client de supervision PC

Étape 3.1 — Client Python

Écris `supervision.py` (base : corrigé TD 08 ex. 3) qui, toutes les secondes : lit `%MW100..104`, affiche un tableau de bord texte, **vérifie le mot de vie**, journalise en CSV, et permet de taper `consigne 650` ou `stop` pour écrire `%MW110/111`.

```
— CUVE — 14:02:31 —
niveau   : 47,2 % ██████████
P1       : MARCHE (123,4 h)   P2 : arrêt (119,0 h)
vanne    : ouverte
défauts  : aucun             vie : OK (12043)
```

Étape 3.2 — Essais croisés (le cœur du TP)

Déroule et documente ces scénarios de bout en bout :

#	Scénario	Attendu
1	Cycle normal observé depuis le PC	niveau qui oscille 40 ↔ 60, alternance visible dans les heures
2	<code>consigne 650</code> depuis le PC	l'arrêt des pompes passe à 65 % ; <code>consigne 950</code> → borné à 80 %
3	Arrêt du simulateur automate	le PC détecte le mot de vie figé < 3 s et l'affiche en alarme
4	<code>stop</code> depuis le PC pendant un remplissage	pompes coupées PAR L'AUTOMATE (le PC n'a écrit qu'une demande)
5	Capteur figé (bloque <code>%IW0.0</code>) pendant une pompe en marche	<code>DEF_CAPTEUR</code> après 30 s, visible côté PC, positions sûres

✅ **Point de contrôle final** : scénario 3 — c'est lui qui distingue une « démo » d'une supervision : un lien TCP ouvert ne prouve pas que l'automate vit.

Étape 3.3 — (option) Version Java

Remplace le client Python par ton programme Java du module 05 §7.1 (j2mod) : même table, même comportement. Constate que la table d'échange documentée rend le langage du client **indifférent** — c'est

exactement son rôle.

Livrables et barème

Livrable	Points
Programme M221 : hystérésis + alternance + chien de garde capteur	/6
Sécurités en aval, prioritaires, testées	/3
Table d'échange documentée, bornage, mot de vie	/4
Client PC complet (lecture, écriture, vie, CSV)	/4
Les 5 essais croisés documentés	/3
Total	/20

Transfert : tu as maintenant vu les deux écosystèmes (TIA et EcoStruxure) sur la même méthode, et construit une chaîne automate ↔ supervision complète — le profil « automaticien qui parle aussi informatique » est très recherché.

FORMATION EMBARQUÉE

TP — tp5 stm32 freertos

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 5 — Station météo STM32 + FreeRTOS (≈ 10 h, en 4 séances)

Objectif pédagogique : porter la station météo du TP 1 sur un environnement professionnel — Nucleo, HAL, drivers écrits maison, DMA, puis architecture multi-tâches FreeRTOS. À la fin, tu as le projet « candidature firmware junior » type.

 **Fiches minutées séance par séance** : [Séance 1 — Socle](#) · [Séance 2 — Driver BME280](#) · [Séance 3 — ADC/DMA + console](#) · [Séance 4 — FreeRTOS](#).

Matériel

- Nucleo-F411RE (ou toute Nucleo-64 ; adapter les broches).
- Capteur **BME280** (I2C — température/humidité/pression, mieux documenté que le DHT22 pour écrire un driver propre) ; à défaut, un potentiomètre simule la « température ».
- STM32CubeIDE installé. Prérequis : module 10 + TD 10 faits.

Discipline du TP : un commit Git par étape verte ; les drivers dans `Core/Src/drivers/` avec leurs `.h` — le `main.c` ne contient jamais d'accès direct à un capteur.

Séance 1 (2 h) — Socle : horloges, UART de debug, GPIO

1. Projet CubeMX pour la carte (Board Selector → périphériques par défaut). Clock tree : SYSCLK à 100 MHz par PLL depuis HSI — note dans ton compte rendu les valeurs M/N/P choisies par l'outil et vérifie $APB1 \leq 50$ MHz.
2. `printf` redirigé vers USART2 (TD 10 ex. 2). Bannière de démarrage avec version (`__DATE__` , `__TIME__`) — réflexe pro : savoir QUEL firmware tourne.
3. LED LD2 en vie (« heartbeat ») : clignotement 1 Hz cadencé par `HAL_GetTick()` (pas de `HAL_Delay` : le socle doit déjà être non bloquant). Bouton B1 par EXTI → inverse la verbosité des logs.

✓ **Point de contrôle 1** : bannière au terminal série, heartbeat régulier, appui bouton pris en compte — et tu sais dire à quelle fréquence tourne chaque bus.

Séance 2 (3 h) — Driver BME280 écrit maison

Interdiction d'utiliser une bibliothèque toute faite : le but EST le driver.

Étape 2.1 — Reconnaissance

I2C1 (PB8/PB9) à 100 kHz. Adresse BME280 : 0x76 ou 0x77 selon câblage — écris d'abord un **scanner I2C** :

```
for (uint8_t a = 1; a < 128; a++)
    if (HAL_I2C_IsDeviceReady(&hi2c1, a << 1, 1, 5) == HAL_OK)
        printf("trouve : 0x%02X\r\n", a);
```

Lis le registre `id` (0xD0) : il doit répondre **0x60**. C'est ton « hello world » I2C.

Étape 2.2 — Le driver

Interface imposée (`drivers/bme280.h`) :

```
typedef struct {
    I2C_HandleTypeDef *i2c;
    uint8_t adresse;           // déjà décalée <<1
    // étalonnage lu en NVM du capteur (datasheet §4.2.2)
    uint16_t dig_T1; int16_t dig_T2, dig_T3;
    /* ... dig_P*, dig_H* ... */
} bme280_t;

bool bme280_init(bme280_t *dev, I2C_HandleTypeDef *i2c, uint8_t addr7);
bool bme280_lire(bme280_t *dev, int32_t *temp_centiemes,
                uint32_t *press_pa, uint32_t *hum_milliemes);
```

Étapes guidées (datasheet Bosch BME280 ouvert à côté) : 1. `init` : vérifier l'id, lire les registres d'étalonnage (0x88..., 0xE1...), configurer `ctrl_hum`, `ctrl_meas`, `config` (oversampling ×1, mode normal). 2. `lire` : lecture **en rafale** des 8 octets 0xF7-0xFE (une seule transaction — les valeurs sont cohérentes entre elles), puis les formules de **compensation en entiers** de la datasheet (§4.2.3 — les recopier est normal, les comprendre est demandé). 3. Sorties en **point fixe** : centièmes de °C, Pa, millièmes de % — aucun `float` dans le driver (module 01 §2.1). 4. Chaque fonction retourne `false` sur échec HAL : le driver ne fait jamais de `printf` et ne bloque jamais plus que ses timeouts I2C.

✓ **Point de contrôle 2** : `T=23.45C P=101325Pa H=48.2%` plausibles au terminal, ET débrancher SDA en cours de route produit des `false` gérés (messages d'erreur côté main, pas de gel).

Séance 3 (2 h) — ADC + DMA et console

1. **ADC1 + DMA circulaire** sur 2 canaux (potentiomètre PA0 = consigne, capteur de température interne du STM32 en canal 16 — comparaison amusante avec le BME280) : tableau `uint16_t adc[2]` rempli en tâche de fond (TD 10 / module 10 §4.6). Moyenne glissante sur 16 échantillons.
2. **Console UART** (reprise du TD 10 ex. 2) avec commandes : `status` (mesures + uptime + version), `log on|off`, `seuil <valeur>`.

✓ **Point de contrôle 3** : pose un point d'arrêt et vérifie dans la vue mémoire que `adc[]` continue de changer pendant que le CPU est arrêté — c'est le DMA, et savoir le *montrer* est le but de l'étape.

Séance 4 (3 h) — Architecture FreeRTOS

Active FreeRTOS (CMSIS-V2) dans CubeMX (base de temps HAL → TIM10 quand l'outil le demande — note pourquoi : SysTick appartient à l'OS).

Architecture imposée

Tâche	Priorité	Période	Rôle
T_Capteur	normale	1 s	lit le BME280, pousse une <code>mesure_t</code> dans <code>q_mesures</code>
T_Traitement	normale	sur message	moyennes, comparaison seuil, pousse vers <code>q_affichage</code>
T_Console	basse	sur caractère	la console série (bloquée sur la queue RX alimentée par l'ISR UART)
T_Heartbeat	la plus basse	500 ms	LED vie + surveillance des piles (<code>uxTaskGetStackHighWaterMark</code>)

Règles : - **Toute communication par queue** (`osMessageQueue`) — aucune variable globale partagée entre tâches (le mutex n'apparaît que si tu partages l'I2C entre deux tâches : explique pourquoi tu ne le fais pas). - L'ISR UART ne fait que pousser l'octet dans une queue (`...FromISR`) — vérifie la priorité NVIC compatible FreeRTOS (module 10 §4.7). - `configCHECK_FOR_STACK_OVERFLOW = 2` pendant la mise au point, et le hook qui `printf` + LED rouge.

Essais de validation

#	Essai	Attendu
1	Fonctionnement 10 min	mesures stables, aucune dérive des piles (high water marks stables)
2	Débrancher le capteur en marche	<code>T_Capteur</code> signale, les autres tâches vivent, reprise au rebranchement
3	Spam de la console (coller 1 Ko de texte)	pas de crash, pas d'octets mélangés dans les logs
4	Point d'arrêt dans <code>T_Traitement</code> 5 s puis reprise	le système rattrape : messages en attente traités, pas de perte silencieuse non expliquée

✓ **Point de contrôle final** : pour l'essai 4, explique dans le compte rendu ce que sont devenus les messages produits pendant la pause (queue pleine ? politique ?) — il n'y a pas de « bonne » réponse unique, il y a une réponse **documentée**.

Livrables et barème

Livrable	Points
Socle : clock tree expliqué, printf, heartbeat non bloquant	/3
Driver BME280 : compensation entière, erreurs gérées, zéro dépendance	/6
ADC+DMA démontré (mesure sous point d'arrêt) + console robuste	/4
Architecture FreeRTOS conforme (queues, ISR minimales, priorités justifiées)	/4
Les 4 essais documentés + README de projet avec schéma	/3
Total	/20

Après ce TP : ton dépôt contient un driver I2C bare-metal, du DMA, une architecture RTOS propre et des essais documentés — mets-le en tête de ton portfolio, c'est exactement ce qu'un recruteur firmware ouvre en premier.

FORMATION EMBARQUÉE

TP1 — Seance 1

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 1 — Fiche de séance 1 : le socle non bloquant (2 h)

En-tête pédagogique

Objectifs	À la fin de la séance tu sais : câbler DHT22/LED/potentiomètre ; écrire des tâches périodiques avec <code>millis()</code> ; gérer une panne de capteur ; appliquer une hystérésis
Prérequis	Module 03 §1-5 lus ; TD 03 exercice 3 fait
Matériel	Uno/Nano, DHT22, LED + 220 Ω , potentiomètre 10 k Ω , breadboard, 6 fils — ou projet Wokwi « Arduino Uno »
Livrable	Un sketch qui mesure/affiche/alerte, robuste au débranchement du capteur, commité sur Git

Déroulé minuté

0:00-0:15 — Câblage et vérification

Composant	Broche composant	Broche Arduino
DHT22	VCC / DATA / GND	5V / D2 / GND (pull-up 10 k Ω entre DATA et VCC si capteur nu)
LED rouge	anode (patte longue) via 220 Ω	D8 ; cathode → GND
Potentiomètre	extrémités / curseur	5V et GND / A0

Vérifie AVANT d'alimenter : aucune LED sans résistance, pas de fil 5V ↔ GND direct. Puis téléverse le sketch vide (`setup` / `loop` vides) pour valider la liaison carte ↔ PC.

0:15-0:30 — Premier contact avec chaque périphérique, séparément

Trois micro-tests de 5 min chacun (c'est une méthode, pas une perte de temps : **on ne débogue jamais deux inconnues à la fois**) :

1. `Serial.println(analogRead(A0));` + `delay(200)` → tourne le potentiomètre : la valeur balaie ~0..1023 ?
2. LED : `digitalWrite(8, HIGH)` → elle s'allume ?
3. DHT : exemple `DHTtester` de la bibliothèque → valeurs plausibles ?

0:30-1:15 — Le squelette à tâches périodiques

Écris (sans copier-coller — tape-le) :

```

#include <DHT.h>
DHT dht(2, DHT22);

uint32_t t_mesure = 0, t_affiche = 0;
float temperature = NAN, humidite = NAN;

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(8, OUTPUT);
}

void loop() {
  uint32_t now = millis();

  if (now - t_mesure >= 2000) {          // tâche 1 : mesurer (2 s)
    t_mesure = now;
    float t = dht.readTemperature(), h = dht.readHumidity();
    if (!isnan(t)) temperature = t;      // on ne garde QUE les lectures valides
    if (!isnan(h)) humidite = h;
  }

  if (now - t_affiche >= 1000) {         // tâche 2 : afficher (1 s)
    t_affiche = now;
    Serial.print(F("T=")); Serial.print(temperature);
    Serial.print(F("C H=")); Serial.print(humidite); Serial.println(F("%"));
  }
}

```

Questions à te poser (réponds par écrit dans ton journal) : - Pourquoi `uint32_t` et pas `int` pour `t_mesure` ? (`int` = 16 bits sur Uno) - Pourquoi `now - t_mesure >= 2000` et pas `now >= t_mesure + 2000` ? (débordement de `millis()` à 49 j : la soustraction reste juste) - Pourquoi `F("T=")` ? (`chaîne en flash` : les 2 Ko de RAM sont précieux)

✅ **Point de contrôle 1 (à 1:15)** : débranche la broche DATA du DHT **en cours de fonctionnement**.

Attendu : l'affichage continue avec les dernières valeurs valides, pas de gel ni de `nan` en boucle. C'est la conséquence directe du `if (!isnan(t))`.

1:15-1:50 — Consigne et hystérésis

Ajoute :

```

const float HYST = 0.5f;
bool alerte = false;

// dans loop(), dans la tâche d'affichage par exemple :
float consigne = 10.0f + analogRead(A0) * (30.0f / 1023.0f); // 10..40 °C

if (!alerte && temperature > consigne + HYST) alerte = true; // bande morte
if (alerte && temperature < consigne - HYST) alerte = false;
digitalWrite(8, alerte);

```

Test : règle la consigne juste au-dessus de la température ambiante, souffle sur le capteur.  **Point de contrôle 2** : la LED bascule franchement, sans clignoter autour du seuil. Puis mets `HYST = 0.0f` et re-observe : tu VOIS maintenant à quoi sert l'hystérésis — remets 0,5.

1:50-2:00 — Commit et journal

```
git add . && git commit -m "TP1 seance 1 : socle non bloquant + hysteresis"
```

Journal de bord (3 lignes minimum) : ce qui a marché du premier coup, ce qui a résisté, ce que tu as compris.

Erreurs fréquentes de cette séance

Symptôme	Cause probable	Remède
<code>nan</code> en permanence	DATA sur le mauvais pin, pull-up absente, lecture < 2 s d'intervalle	vérifier câblage, espacer les lectures
Valeurs A0 qui « flottent »	curseur du potentiomètre non branché sur A0	relire le brochage
LED toujours éteinte	LED à l'envers (polarité) ou résistance vers le mauvais rail	inverser la LED
L'affichage se fige	un <code>delay()</code> s'est glissé quelque part	interdit hors setup !
Caractères illisibles au moniteur	vitesse $\neq$ 115200 dans le moniteur série	aligner les deux

Travail à la maison (30 min)

Ajoute une **troisième tâche périodique** (500 ms) qui fait clignoter la LED intégrée (D13) — un « battement de cœur » qui prouve que la boucle n'est jamais bloquée. Il te faudra une deuxième variable d'horodatage : constate que les trois périodes cohabitent sans s'influencer.

 Fiche suivante : [Séance 2 — OLED et architecture](#)

FORMATION EMBARQUÉE

TP1 — Seance 2

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 1 — Fiche de séance 2 : OLED et architecture en modules (3 h)

En-tête pédagogique

Objectifs	Piloter un écran I2C ; découper un projet Arduino en modules <code>.h/.cpp</code> ; comprendre « interface vs implémentation » en pratique
Prérequis	Séance 1 finie et commitée ; module 02 §2 (classes) lu
Matériel	Montage de la séance 1 + OLED SSD1306 128×64 I2C
Livrable	Le même comportement qu'en séance 1, affiché sur OLED, avec un <code>.ino</code> < 60 lignes et 3 modules

Déroulé minuté

0:00-0:20 — Câblage OLED et scanner I2C

OLED : VCC → 5V (3,3 V selon module), GND → GND, SDA → A4, SCL → A5.

Avant toute bibliothèque d'écran, exécute un **scanner I2C** (croquis d'exemple « Wire → i2c_scanner » de l'IDE). Attendu : `0x3C` détecté. Méthode : *on valide le bus avant de déboguer l'écran* — si le scanner ne voit rien, aucune bibliothèque ne marchera (SDA/SCL inversés dans 80 % des cas).

0:20-0:50 — Premier affichage

Installe `Adafruit SSD1306` (+ `Adafruit GFX`). Mini-test isolé :

```
#include <Wire.h>
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 oled(128, 64, &Wire, -1);

void setup() {
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  oled.clearDisplay();
  oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(2);
  oled.setCursor(0, 0);
  oled.print(F("Bonjour"));
  oled.display();           // RIEN ne s'affiche sans display() !
}

void loop() {}
```

Piège n°1 de l'OLED : oublier `oled.display()` (tout se dessine dans un tampon en RAM, `display()` l'envoie à l'écran).

0:50-2:20 — LA refactorisation en modules (le cœur de la séance)

Objectif : le `.ino` ne doit plus contenir QUE l'orchestration. Crée trois paires de fichiers (IDE : bouton « ... » → Nouvel onglet) :

capteur.h — l'interface (ce que les autres ont le droit de savoir) :

```
#ifndef CAPTEUR_H
#define CAPTEUR_H
#include <stdint.h>

void capteur_init();
// À appeler à chaque tour : fait une mesure si l'intervalle est écoulé.
void capteur_tick(uint32_t maintenant_ms);
// Dernières valeurs valides (NAN si jamais rien reçu)
float capteur_temperature();
float capteur_humidite();
bool capteur_en_panne();          // 3 échecs consécutifs
#endif
```

capteur.cpp — l'implémentation (SEUL fichier qui connaît le DHT) :

```
#include "capteur.h"
#include <DHT.h>

static DHT dht(2, DHT22);          // static : invisible hors du module
static float t_ = NAN, h_ = NAN;
static uint8_t echecs_ = 0;
static uint32_t derniere_ = 0;

void capteur_init() { dht.begin(); }

void capteur_tick(uint32_t now) {
    if (now - derniere_ < 2000) return;
    derniere_ = now;
    float t = dht.readTemperature(), h = dht.readHumidity();
    if (isnan(t) || isnan(h)) {
        if (echecs_ < 255) echecs_++;
    } else {
        echecs_ = 0;
        t_ = t; h_ = h;
    }
}

float capteur_temperature() { return t_; }
float capteur_humidite()    { return h_; }
bool capteur_en_panne()     { return echecs_ >= 3; }
```

affichage.h/.cpp : même principe — `affichage_init()`, `affichage_tick(now, t, h, consigne, alerte, panne)` qui rafraîchit l'OLED au plus 5×/s. Personne d'autre n'inclut `Adafruit_SSD1306.h`.

alerte.h/.cpp : `alerte_tick(now, temperature, consigne)` qui rend l'état avec hystérésis et pilote la LED.

Le `.ino` final (c'est TOUT ce qu'il doit rester) :

```
#include "capteur.h"
#include "affichage.h"
#include "alerte.h"

void setup() {
    Serial.begin(115200);
    capteur_init();
    affichage_init();
    alerte_init();
}

void loop() {
    uint32_t now = millis();
    float consigne = 10.0f + analogRead(A0) * (30.0f / 1023.0f);

    capteur_tick(now);
    bool alarme = alerte_tick(now, capteur_temperature(), consigne);
    affichage_tick(now, capteur_temperature(), capteur_humidite(),
                  consigne, alarme, capteur_en_panne());
}
```

✓ **Point de contrôle (2:20)** — les trois critères d'architecture : 1. Le `.ino` fait < 60 lignes et n'inclut aucune bibliothèque matérielle. 2. `grep -r "Adafruit" *.ino` ne renvoie rien. 3. Question test : « pour remplacer l'OLED par un LCD 16×2, quels fichiers changent ? » — la réponse doit être : `affichage.cpp` (et lui seul).

2:20-2:50 — L'écran affiche l'état de panne

Utilise `capteur_en_panne()` pour afficher « CAPTEUR HS » en gros à l'écran (et rejoue le test du débranchement de la séance 1). Un système embarqué **montre** ses pannes — un écran qui affiche une vieille valeur sans le dire est un mensonge.

2:50-3:00 — Commit

```
git add . && git commit -m "TP1 seance 2 : OLED + architecture en modules"
```

Erreurs fréquentes

Symptôme	Cause	Remède
Écran blanc/noir mais scanner OK	oubli de <code>display()</code> , mauvaise taille (128×32 vs 64)	vérifier le constructeur
<code>multiple definition of...</code> à l'édition de liens	variable définie dans un <code>.h</code> inclus deux fois	définir dans le <code>.cpp</code> , <code>extern</code> ou accesseur dans le <code>.h</code>
Rien ne compile après découpage	garde d'inclusion oubliée, <code>#include</code> circulaire	<code>#ifndef/#define/#endif</code> partout
L'affichage « rame »	<code>display()</code> appelé à chaque tour de loop	le limiter à 5 Hz (tâche périodique)
RAM presque pleine à la compilation	chaînes sans <code>F()</code> , tampon OLED (1 Ko incompressible)	<code>F()</code> partout ; c'est l'OLED qui coûte

Travail à la maison (45 min)

Réécris `alerte` en classe **C++** (constructeur prenant broche + hystérésis, méthode `tick()`) au lieu de fonctions + `static` . Compare les deux styles dans ton journal : qu'est-ce que la classe apporte ici ?
(réponse attendue : plusieurs instances possibles, état impossible à corrompre de l'extérieur — cf. TD 02.)

→ Fiche suivante : [Séance 3 — FSM d'interface et journalisation](#)

FORMATION EMBARQUÉE

TP1 — Seance 3

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 1 — Fiche de séance 3 : FSM d'interface, appui long, journalisation (3 h)

En-tête pédagogique

Objectifs	Anti-rebond en pratique ; distinguer appui court/long ; FSM d'interface multi-pages ; min/max ; (option) écrire sur carte SD en tolérant son absence
Prérequis	Séance 2 finie ; TD 03 exercices 2-3
Matériel	Montage séance 2 + bouton poussoir sur D3 (+ module SD SPI en option)
Livrable	Interface 3 pages naviguée au bouton, RAZ min/max par appui long

Déroulé minuté

0:00-0:30 — Module bouton avec appui court / appui long

Nouveau module `bouton.h/.cpp`. Spécification (écris-la d'abord !) : - anti-rebond 20 ms (TD 03) ; - `bouton_evenement()` retourne `AUCUN`, `COURT` (relâché avant 2 s) ou `LONG` (maintenu ≥ 2 s ; l'événement part au franchissement des 2 s, pas au relâchement — meilleure sensation utilisateur).

```
// bouton.h
enum class EvtBouton : uint8_t { AUCUN, COURT, LONG };
void bouton_init();
EvtBouton bouton_tick(uint32_t maintenant_ms);
```

```
// bouton.cpp — FSM à 3 états : RELACHE, APPUYE, LONG_SIGNALE
#include "bouton.h"
#include <Arduino.h>

static const uint8_t BROCHE = 3;
static const uint16_t DEBOUNCE_MS = 20;
static const uint16_t LONG_MS = 2000;

enum class Etat : uint8_t { RELACHE, APPUYE, LONG_SIGNALE };
static Etat etat = Etat::RELACHE;
static bool brut_prec = false;
static uint32_t t_chgt = 0, t_appui = 0;
static bool stable = false;

void bouton_init() { pinMode(BROCHE, INPUT_PULLUP); }

EvtBouton bouton_tick(uint32_t now) {
    bool brut = (digitalRead(BROCHE) == LOW);
    if (brut != brut_prec) { t_chgt = now; brut_prec = brut; }
    if (now - t_chgt > DEBOUNCE_MS) stable = brut;      // état filtré

    switch (etat) {
    case Etat::RELACHE:
        if (stable) { etat = Etat::APPUYE; t_appui = now; }
        break;
    case Etat::APPUYE:
        if (now - t_appui >= LONG_MS) { etat = Etat::LONG_SIGNALE; return EvtBouton::LONG; }
        if (!stable) { etat = Etat::RELACHE; return EvtBouton::COURT; }
        break;
    case Etat::LONG_SIGNALE:           // on attend le relâchement
        if (!stable) etat = Etat::RELACHE; // sans générer d'événement
        break;
    }
    return EvtBouton::AUCUN;
}
```

Remarque de conception : le bouton est LUI-MÊME une petite FSM — dessine-la dans ton journal (3 états, 4 transitions). **Test** : `Serial.println` de chaque événement ; vérifie qu'un appui long ne génère PAS un court au relâchement (c'est le rôle de `LONG_SIGNALE`).

0:30-1:00 — Min/max dans le module capteur

Ajoute à `capteur.h` : `capteur_temp_min()` , `capteur_temp_max()` , `capteur_raz_minmax()` .

Implémentation : mise à jour à chaque mesure valide (attention à l'initialisation : partir de la première mesure, pas de 0 — un min initialisé à 0 est faux dans une pièce à 20 °C ; utilise NAN comme sentinelle).

1:00-2:00 — La FSM de pages

Dans le `.ino` (c'est de l'orchestration, donc c'est sa place) :

```
enum class Page : uint8_t { MESURES, MINMAX, CONSIGNE };
Page page = Page::MESURES;

// dans loop() :
EvtBouton evt = bouton_tick(now);
if (evt == EvtBouton::COURT) {
    page = (page == Page::MESURES) ? Page::MINMAX
        : (page == Page::MINMAX) ? Page::CONSIGNE
        : Page::MESURES;
}
if (evt == EvtBouton::LONG && page == Page::MINMAX) {
    capteur_raz_minmax(); // RAZ seulement depuis la page concernée
}
```

Et `affichage_tick` prend maintenant la page en paramètre et dessine la bonne vue (3 fonctions statiques internes : `dessiner_mesures()`, `dessiner_minmax()`, `dessiner_consigne()`).

✓ **Point de contrôle (2:00)** : navigation fluide pendant que les mesures continuent ; appui long sur la page MIN/MAX → valeurs remises à la mesure courante ; appui long ailleurs → rien (comportement **spécifié**, pas accidentel).

2:00-2:50 — (option matériel) Journalisation SD

Module `journal.h/.cpp` avec la carte SD (CS=D10, bibliothèque `SD`) : - toutes les 60 s : ligne CSV `millis;temperature;humidite` dans `METEO.CSV` ; - **l'absence de carte est un état normal** : `journal_ok()` renvoie false, l'écran ajoute un pictogramme, et le module retente l'init toutes les 30 s. Rien ne bloque, rien ne plante.

Sans matériel SD : écris le même module mais vers `Serial` (préfixe `CSV;`) — l'architecture est le vrai objectif.

2:50-3:00 — Commit + journal de bord

```
git add . && git commit -m "TP1 seance 3 : FSM pages, appui long, journal"
```

Erreurs fréquentes

Symptôme	Cause	Remède
Un appui = 2 ou 3 changements de page	anti-rebond absent/trop court, ou événement généré sur niveau au lieu de front	revoir la FSM bouton
L'appui long déclenche aussi un court	pas d'état <code>LONG_SIGNALE</code>	voir le corrigé ci-dessus
min/max absurdes (0 ou -273)	initialisation à 0 au lieu de la 1 ^{re} mesure	sentinelle NAN
Init SD qui gèle 2 s toute la boucle	<code>SD.begin()</code> est bloquant	ne le retenter que toutes les 30 s, hors du chemin chaud
Plus de RAM (Uno)	SD + OLED + chaînes	<code>F()</code> partout ; c'est le moment de mesurer avec l'IDE

Travail à la maison (30 min)

Sur papier : dessine le **diagramme d'états complet** de ton application (pages × événements) et la FSM du bouton. Range les deux dessins dans le dépôt (photo `docs/fsm.jpg`) — un firmware dont la FSM n'est pas dessinée n'est pas fini.

➡ Fiche suivante : [Séance 4 — ESP32 et MQTT](#)

FORMATION EMBARQUÉE

TP1 — Seance 4

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 1 — Fiche de séance 4 : portage ESP32 et MQTT (4 h)

En-tête pédagogique

Objectifs	Porter un projet multi-modules sur une autre carte ; se connecter en Wi-Fi ; publier en MQTT avec reconnexion propre ; vérifier de bout en bout
Prérequis	Séances 1-3 finies ; module 03 §7 étape 6
Matériel	ESP32 DevKit + DHT22 + OLED (ou projet Wokwi « ESP32 ») ; sur PC : <code>mosquitto-clients</code> ou le client web HiveMQ
Livrable	La station publie ses mesures en JSON toutes les 10 s sur un broker public, et survit aux coupures réseau

Déroulé minuté

0:00-0:40 — Portage matériel et logiciel

Nouveau câblage (3,3 V ! le DHT22 accepte, vérifie ton OLED) :

Élément	Uno (avant)	ESP32 (après)
DHT22 DATA	D2	GPIO 4
OLED SDA/SCL	A4/A5	GPIO 21/22 (I2C par défaut)
LED alerte	D8	GPIO 2 (LED carte sur beaucoup de modules)
Bouton	D3	GPIO 15
Potentiomètre	A0	GPIO 34 (entrée seule, ADC1)

Dans l'IDE : installer le support ESP32 (gestionnaire de cartes), carte « ESP32 Dev Module ». Adaptations de code attendues **uniquement** dans les constantes de broches des modules (c'est le test de l'architecture de la séance 2) + `anaLogRead` qui renvoie 0..4095 (12 bits) → adapter la mise à l'échelle de la consigne.

✅ **Point de contrôle 1 (0:40)** : la station de la séance 3 tourne à l'identique sur ESP32. Note dans ton journal les 4-5 lignes qui ont changé — si tu as dû toucher `capteur.cpp` au-delà du numéro de broche, remonte la cause.

0:40-1:30 — Module réseau : Wi-Fi + MQTT

Nouveau module `reseau.h/.cpp` (bibliothèque `PubSubClient`) :

```
// reseau.h
void reseau_init(const char* ssid, const char* mdp,
                const char* broker, const char* id_client);
void reseau_tick(uint32_t maintenant_ms);    // entretient la connexion
bool reseau_connecte();
bool reseau_publier(const char* topic, const char* json);
```

```
// reseau.cpp – les points délicats
#include "reseau.h"
#include <WiFi.h>
#include <PubSubClient.h>

static WiFiClient net;
static PubSubClient mqtt(net);
static const char *ssid_, *mdp_, *id_;
static uint32_t derniere_tentative = 0;

void reseau_init(const char* ssid, const char* mdp,
                const char* broker, const char* id_client) {
    ssid_ = ssid; mdp_ = mdp; id_ = id_client;
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid_, mdp_);    // NON bloquant : on n'attend pas ici
    mqtt.setServer(broker, 1883);
}

void reseau_tick(uint32_t now) {
    if (WiFi.status() != WL_CONNECTED) return;    // le Wi-Fi retente seul

    if (!mqtt.connected()) {
        // Reconnexion au plus 1 fois / 5 s : JAMAIS en rafale
        if (now - derniere_tentative >= 5000) {
            derniere_tentative = now;
            mqtt.connect(id_);
        }
        return;
    }
    mqtt.loop();    // entretien : à appeler très souvent
}

bool reseau_connecte() { return WiFi.status() == WL_CONNECTED && mqtt.connected(); }

bool reseau_publier(const char* topic, const char* json) {
    return reseau_connecte() && mqtt.publish(topic, json);
}
```

Points de conception à retenir : - **Aucune attente bloquante** : pas de `while (WiFi.status() != ...)` — la station mesure et affiche MÊME sans réseau (le réseau est un service, pas une condition de vie). - Reconnexion **espacée** (5 s) : un broker public bannit les clients qui martèlent. - **id_client unique** (mets ton prénom + un nombre) : deux clients de même id se déconnectent mutuellement en boucle — LE grand classique des brokers publics.

1:30-2:15 — Publication JSON

Dans le `.ino`, nouvelle tâche périodique (10 s) :

```
if (now - t_publie >= 10000 && !capteur_en_panne()) {
  t_publie = now;
  char json[96];
  snprintf(json, sizeof json,
    "{\"t\":%.1f,\"h\":%.1f,\"alerte\":\"%s\",
    capteur_temperature(), capteur_humidite(),
    alarme ? "true" : "false"}");
  reseau_publier("formation/TONPRENOM/meteo", json);
}
```

Ajoute l'état réseau sur l'OLED (icône ou « WiFi/MQTT/-- »).

Wokwi : SSID `Wokwi-GUEST`, mot de passe vide ; broker `broker.hivemq.com`.

2:15-3:00 — Vérification de bout en bout

Sur PC :

```
mosquitto_sub -h broker.hivemq.com -t "formation/TONPRENOM/#" -v
```

(ou le client web <http://www.hivemq.com/demos/websocket-client/>).

✓ **Point de contrôle 2** : les JSON arrivent toutes les 10 s. Puis les trois pannes à provoquer et documenter :

Panne provoquée	Comportement attendu
Coupure Wi-Fi (désactive le routeur / bouton Wokwi)	station vivante, OLED indique « -- », reprise SEULE au retour
Capteur débranché	plus de publication (on ne publie pas du NAN), écran « CAPTEUR HS », le reste vit
Broker inaccessible (mauvais nom 2 min)	tentatives espacées de 5 s visibles, pas de gel

3:00-3:45 — Consommer les données (aperçu du module 05)

Petit consommateur en Python sur PC (10 lignes, `pip install paho-mqtt`) :

```
import json, paho.mqtt.client as mqtt

def sur_message(cli, _, msg):
    m = json.loads(msg.payload)
    print(f"{msg.topic} : {m['t']} °C, {m['h']} %"
          + (" *** ALERTE ***" if m.get("alerte") else ""))

cli = mqtt.Client()
cli.on_message = sur_message
cli.connect("broker.hivemq.com", 1883)
cli.subscribe("formation/TONPRENOM/meteo")
cli.loop_forever()
```

Tu viens de construire une chaîne IoT complète : capteur → firmware C++ → Wi-Fi → broker → application PC. (La version Java robuste est l'objet du mini-projet du module 05.)

3:45-4:00 — Commit final + auto-évaluation

```
git add . && git commit -m "TP1 seance 4 : ESP32 + MQTT avec reconnexion"
```

Remplis la grille /20 de la fiche TP principale ([tp1-arduino-station-meteo.md](#) §grille), **avec preuves** (captures du moniteur, de `mosquitto_sub`, du journal Git).

Erreurs fréquentes

Symptôme	Cause	Remède
Déconnexions MQTT en boucle	id client non unique sur broker public	id avec prénom + aléa
<code>Brownout detector triggered</code> (ESP32 redémarre)	alimentation USB faiblarde au pic Wi-Fi	autre câble/port, condensateur 470 µF
OLED muette après portage	OLED 5 V sur bus 3,3 V, ou mauvais pins I2C	vérifier module et GPIO 21/22
<code>analogRead</code> plafonne bizarrement	GPIO 34-39 = entrée seule OK, mais ADC2 inutilisable avec Wi-Fi	rester sur ADC1 (GPIO 32-39)
Publications qui cessent après des heures	tas fragmenté par des <code>String</code>	<code>snprintf</code> + tampons fixes (déjà le cas ici)

Pour aller plus loin (hors séance)

Trois extensions, chacune = un commit de portfolio : **OTA** (mise à jour par Wi-Fi, bibliothèque ArduinoOTA) ; **deep sleep** entre mesures (autonomie batterie ×50, mais il faut repenser l'OLED et le bouton — bon exercice de conception) ; **Node-RED** sur PC pour tracer les courbes et déclencher un e-mail d'alerte.

FORMATION EMBARQUÉE

TP2 — Seance 1

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 2 — Fiche de séance 1 : spécification et émetteur UART (2 h)

En-tête pédagogique

Objectifs	Écrire une spécification chiffrée avant le code ; calculer une erreur de baudrate ; coder et simuler un émetteur UART ; mesurer au chronogramme
Prérequis	Module 04 en entier ; TD 04 exercices 1-2 (mux, compteur) réussis en simulation
Outils	GHDL + GTKWave installés (<code>sudo apt install ghdl gtkwave</code>) ou edaplayground.com
Livrable	<code>uart_tx.vhd</code> + testbench, durée de bit vérifiée au curseur : 434 cycles pile

Déroulé minuté

0:00-0:10 — Préparer l'atelier

Crée un dossier de travail avec le script de simulation :

```
mkdir tp2-uart && cd tp2-uart && git init
```

```
#!/bin/sh
# simule.sh <sources...> <testbench.vhd> - analyse, élabore, simule, affiche
set -e
ghdl -a --std=08 "$@"
TB=$(basename "${@: -1}" .vhd)
ghdl -e --std=08 "$TB"
ghdl -r --std=08 "$TB" --wave="$TB.ghw" --stop-time=5ms
gtkwave "$TB.ghw" &
```

`chmod +x simule.sh` . Vérifie l'installation avec le mux du TD 04 : `./simule.sh mux4.vhd tb_mux4.vhd` doit ouvrir GTKWave.

0:10-0:40 — La spécification (papier, PAS de code)

Remplis ce tableau dans un fichier `SPEC.md` — c'est un livrable noté :

Paramètre	Valeur	Justification
Format	8N1	1 start + 8 données (LSB d'abord) + 1 stop
Baudrate	115 200	standard de debug
F horloge	50 MHz	générique, adaptable
Ticks par bit	$\lfloor 50\,000\,000 / 115\,200 \rfloor = \mathbf{434}$	division entière
Baud réel	$50\,000\,000 / 434 \approx \mathbf{115\,207}$	
Erreur par bit	$(115\,207 - 115\,200) / 115\,200 \approx \mathbf{+0,006\%}$	
Dérive sur 10 bits	$\approx 0,06\%$ de la durée de trame	$\ll \frac{1}{2}$ bit (5 %) : OK

Les calculs à savoir refaire de tête en entretien : 1. **Ticks par bit** = $f_{\text{clk}} / \text{baud}$ (division entière → erreur d'arrondi). 2. **La limite** : le récepteur échantillonne au milieu du bit ; l'erreur cumulée au 10^e bit doit rester $< \frac{1}{2}$ bit, soit $\sim 5\%$ par bit → un couple horloge/baud qui donne $> \sim 2\%$ d'erreur d'arrondi est à proscrire. 3. Contre-exemple à calculer : $1\text{ MHz} / 115\,200 = 8,68$ → arrondi à 8 ou 9, erreur ≈ 8 ou 4% → **hors spec**. C'est pour ça que les vieux quartz « UART-friendly » font $1,8432\text{ MHz}$ ($= 16 \times 115\,200$).

Dessine aussi la trame 8N1 pour l'octet 0x55 (repos haut, start bas, **1,0,1,0,1,0,1,0** LSB d'abord, stop haut) — tu la compareras au chronogramme à 1:50.

0:40-1:30 — Coder l'émetteur

Écris `uart_tx.vhd` **de mémoire** en t'appuyant sur la FSM 4 états (REPOS → START → DONNEES → STOP). Ne regarde le corrigé (`code/vhdl/uart_tx.vhd`) qu'en cas de blocage > 15 min. Les 4 pièges qui te guettent :

1. **Capturer** `tx_data` **dans un registre interne** à l'acceptation du `tx_start` — sinon l'appelant peut changer la donnée en pleine émission.
2. LSB d'abord : `tx <= registre(idx_bit)` avec `idx_bit` qui monte de 0 à 7.
3. Le compteur de ticks se compare à `TICKS_PAR_BIT - 1` (le « off by one » classique : compter de 0 à N-1 fait N cycles).
4. `busy` levé de START à STOP inclus.

1:30-1:50 — Le testbench

Reprends la structure du TD 04 exercice 5 : horloge 50 MHz (`clk <= not clk after 10 ns;`), impulsion `tx_start` d'un cycle, envoi de `x"55"`, attente de `busy = '0'`.

```
./simule.sh uart_tx.vhd tb_uart_tx.vhd
```

1:50-2:00 — Mesure au chronogramme (✓ point de contrôle)

Dans GTKWave : ajoute `tx`, `busy`, `etat`, `cpt_tick`. Avec les **deux curseurs** (clic + clic-milieu), mesure : - durée du bit de start : attendu **8 680 ns** ($434 \times 20\text{ ns}$) pile ; - la séquence des bits de données pour 0x55 : **1,0,1,0,1,0,1,0** — compare avec ton dessin de 0:40 ; - durée totale de trame : $10 \times 8\,680 = \mathbf{86,8\ \mu s}$.

Un écart d'un cycle (8 660 ou 8 700 ns) = piège n°3 ci-dessus : corrige avant de continuer, le récepteur de la séance 2 ne pardonnera pas.

Commit : `git add . && git commit -m "TP2 seance 1 : spec + uart_tx valide"`.

Erreurs fréquentes

Symptôme	Cause	Remède
GTKWave vide	simulation stoppée avant les stimuli (<code>--stop-time</code> trop court) ou pas de <code>wait</code> final	vérifier la console GHDL
<code>tx</code> reste à 'U'	<code>tx</code> affecté seulement dans certains états au lieu de tous	l'affecter dans chaque branche du case
Trame trop courte d'un bit	transition d'état ET incrément de compteur le même cycle mal ordonnés	revoir : remise à zéro du compteur à chaque changement d'état
Le 2 ^e envoi émet l'ancienne donnée	<code>tx_data</code> lu pendant DONNEES au lieu d'être capturé	piège n°1
<code>ghdl: cannot find entity</code>	ordre d'analyse : les sources AVANT le testbench	<code>./simule.sh sources... tb.vhd</code>

Travail à la maison (30 min)

Ajoute un generic `PARITE : boolean := false` qui, à vrai, insère un bit de parité paire entre le bit 7 et le stop (XOR de tous les bits de données — en VHDL : `xor_reduce` ou une boucle). Testbench : vérifie la trame de 0x55 (parité 0) et de 0x54 (parité 1).

➡ Fiche suivante : [Séance 2 — Le récepteur](#)

FORMATION EMBARQUÉE

TP2 — Seance 2

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 2 — Fiche de séance 2 : le récepteur UART (3h)

En-tête pédagogique

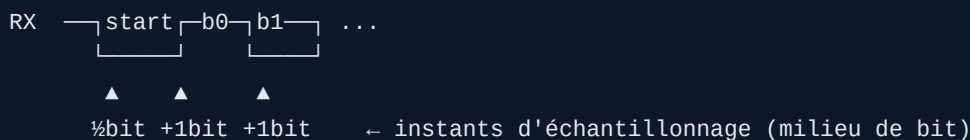
Objectifs	Synchroniser une entrée asynchrone ; sur-échantillonner ; échantillonner au milieu du bit ; valider par test en boucle exhaustif
Prérequis	Séance 1 : <code>uart_tx</code> validé au chronogramme
Outils	GHDL + GTKWave
Livrable	<code>uart_rx.vhd</code> + <code>tb_uart_boucle.vhd</code> affichant « BOUCLE OK, 256/256 octets »

Déroulé minuté

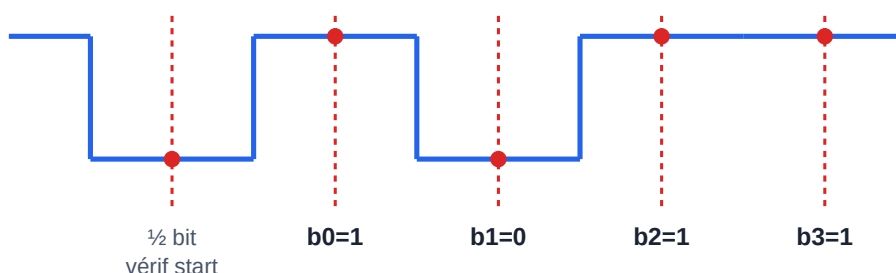
0:00-0:30 — Comprendre le problème (schéma avant code)

Le récepteur n'a **pas** l'horloge de l'émetteur. Il doit : 1. Détecter le front descendant du start (la ligne au repos est à 1). 2. Attendre $\frac{1}{2}$ bit et revérifier : toujours à 0 ? → vrai start ; sinon c'était un parasite (filtre anti-glitch **gratuit**). 3. Ensuite, attendre **1 bit** entre chaque échantillon → on tombe pile au milieu de chaque bit de donnée, là où le signal est le plus stable.

Dessine ce chronogramme dans ton journal :



Récepteur UART — échantillonnage au MILIEU de chaque bit



On se resynchronise sur le front du start, on attend $\frac{1}{2}$ bit (revérif = anti-glitch), puis 1 bit entre chaque point.
Au milieu, le signal est stable → marge maximale contre le désaccord d'horloge entre émetteur et récepteur.

Échantillonnage au milieu de chaque bit côté récepteur

Pourquoi le milieu et pas le bord ? Aux bords, le signal transitionne (temps de montée, désynchronisation résiduelle) : c'est là qu'on se trompe. Au milieu, on a la marge maximale de part et d'autre.

0:30-0:45 — Le synchroniseur : non négociable

Toute entrée asynchrone (`rx` vient d'un autre domaine d'horloge) passe par **2 bascules en série** avant utilisation :

```
sync0 <= rx;      -- peut être métastable un court instant
sync1 <= sync0;   -- stabilisé : on n'utilise QUE sync1 ensuite
```

Sans ça, un changement de `rx` pile sur le front d'horloge peut mettre une bascule dans un état intermédiaire (métastabilité) qui se propage et corrompt toute la FSM. C'est LE réflexe qu'un correcteur d'entretien vérifie en premier.

0:45-1:45 — Coder `uart_rx`

FSM 4 états : REPOS → VERIF_START → DONNEES → STOP. Pars du squelette du TP principal (§2.2) et complète DONNEES et STOP. Points de vigilance :

- Dans DONNEES : à chaque fois que `cpt = TPB - 1`, échantillonner `shreg(idx) <= sync1` (LSB d'abord), puis incrémenter `idx` ou passer à STOP après le bit 7.
- Dans STOP : au milieu du bit de stop, si `sync1 = '1'` → trame valide : `donnee <= shreg; valide <= '1'`. Sinon (stop à 0 = erreur de trame), jeter silencieusement et revenir à REPOS.
- `valide` est une **impulsion d'un cycle** : mets `valide <= '0'` ; en tête de process (valeur par défaut), ne le lève que dans STOP.

Corrigé complet de référence : `code/vhdl/uart_rx.vhd` — mais essaie sérieusement avant de l'ouvrir.

1:45-2:30 — Le testbench en boucle : le juge de paix

Le test le plus élégant : brancher TX → RX et vérifier que ce qui sort égale ce qui entre, **pour les 256 octets possibles**. Copie `code/vhdl/tb_uart_boucle.vhd` ou réécris-le : une boucle `for i in 0 to 255`, envoi, `wait until valide`, `assert d_out = d_in severity failure`.

```
ghdl -a --std=08 uart_tx.vhd uart_rx.vhd tb_uart_boucle.vhd
ghdl -e --std=08 tb_uart_boucle
ghdl -r --std=08 tb_uart_boucle --stop-time=100ms
```

✓ **Point de contrôle** : la console affiche `tb_uart_boucle : BOUCLE OK, 256/256 octets`. (C'est exactement le résultat obtenu et vérifié dans ce dépôt.)

2:30-2:55 — Déboguer un échec (si besoin)

Si un octet précis échoue (`assert failure` avec le numéro) : relance en générant les ondes et zoome sur cet octet.

```
ghdl -r --std=08 tb_uart_boucle --wave=rx.ghw --stop-time=100ms
gtkwave rx.ghw
```

Ajoute `etat`, `cpt`, `idx`, `shreg`, `sync1`. 90 % des bugs sont un `cpt = TPB` au lieu de `TPB - 1` (échantillonnage un cycle trop tard, qui dérive et rate le dernier bit). Regarde si l'instant d'échantillonnage glisse vers le bord du bit au fil des 8 bits.

```
git add . && git commit -m "TP2 seance 2 : uart_rx, boucle 256/256 verte"
```

Erreurs fréquentes

Symptôme	Cause	Remède
~1 octet sur 2 faux	pas de synchroniseur, ou échantillonnage au bord	2 bascules + échantillonner au milieu
Le dernier bit (b7) toujours faux	dérive « off by one » du compteur	<code>cpt = TPB - 1</code> , pas <code>TPB</code>
<code>valide</code> reste à 1	pas de valeur par défaut en tête de process	<code>valide <= '0'</code> ; avant le case
Reçoit un octet décalé (bits inversés)	MSB d'abord au lieu de LSB	<code>shreg(idx)</code> avec idx croissant de 0
Boucle infinie en simu	<code>wait until valide='1'</code> mais valide ne monte jamais	vérifier la logique de STOP

Travail à la maison (45 min)

Ajoute une sortie `erreur_trame : std_logic` (impulsion) levée quand le bit de stop vaut 0. Fabrique un stimulus qui envoie une trame corrompue (force `fil <= '0'` au moment du stop) et vérifie que `erreur_trame` se lève et que `valide` ne se lève pas.

→ Fiche suivante : [Séance 3 — Robustesse et intégration](#)

FORMATION EMBARQUÉE

TP2 — Seance 3

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 2 — Fiche de séance 3 : robustesse et module d'écho (3 h)

En-tête pédagogique

Objectifs	Tester aux limites (glitch, horloges désaccordées) ; assembler un système hiérarchique ; gérer un conflit de ressource ; (option) passer sur carte réelle
Prérequis	Séances 1-2 : boucle 256/256 verte
Outils	GHDL + GTKWave ; (option) Vivado/Quartus + carte
Livrable	3 tests de robustesse passés + module d'écho fonctionnel

Déroulé minuté

0:00-0:45 — Test 1 : le glitch anti-parasite

Ajoute au testbench un stimulus qui force un faux start hors trame :

```
-- entre deux octets, un parasite plus court qu'un demi-bit
fil <= '0'; wait for 3 us;    -- 3 µs < demi-bit (4,34 µs) : doit être ignoré
fil <= '1'; wait for 20 us;
```

Attendu : `valide` ne se lève JAMAIS pour ce parasite. C'est la vérification à mi-start (séance 2) qui fait le travail. Observe au chronogramme : la FSM va en VERIF_START puis **retourne à REPOS** car `sync1` est remonté à 1 au milieu.

0:45-1:45 — Test 2 : horloges désaccordées (le vrai test réel)

Dans la vraie vie, l'émetteur et le récepteur ont des quartz légèrement différents. Instancie-les avec des `F_CLK` distincts :

```
emetteur : entity work.uart_tx
  generic map (F_CLK => 50_000_000, BAUD => 115_200) port map (...);
recepteur : entity work.uart_rx
  generic map (F_CLK => 49_000_000, BAUD => 115_200) port map (...);
-- -2 % : le récepteur croit compter 50 MHz mais l'horloge fait 49
```

À **-2 %** : la boucle 256/256 doit encore passer (marge du milieu de bit). À **-5 %** : elle échoue — et c'est le but. Calcule pourquoi : erreur de 5 % par bit $\times$ $\sim 9,5$ bits (du milieu du start au milieu du bit 7) ≈ 47 % d'un bit $\rightarrow$ on frôle le bord, un octet finit par se décaler. **Documente ce calcul** : c'est la démonstration que le sur-échantillonnage a une limite quantifiable, pas magique.

Note pratique : ici les deux entités partagent la MÊME horloge de simulation ; le désaccord est *émulé* par le `F_CLK` que le récepteur croit avoir. Pour un désaccord physique réel, on générerait deux

horloges de périodes différentes — fais-le en bonus si tu veux être rigoureux.

1:45-2:30 — Module d'écho hiérarchique

Assemble un `top_echo` qui renvoie chaque octet reçu :

```
architecture rtl of top_echo is
    signal rx_data : std_logic_vector(7 downto 0);
    signal rx_valide, tx_busy, tx_start : std_logic;
    signal registre_attente : std_logic_vector(7 downto 0);
    signal octet_en_attente : std_logic := '0';
begin
    u_rx : entity work.uart_rx port map (clk=>clk, rx=>rx,
                                         donnee=>rx_data, valide=>rx_valide);
    u_tx : entity work.uart_tx port map (clk=>clk, tx_start=>tx_start,
                                         tx_data=>registre_attente,
                                         tx=>tx, busy=>tx_busy);

    -- Gestion du cas "TX occupé quand un octet arrive" : un registre d'attente
    process (clk)
    begin
        if rising_edge(clk) then
            tx_start <= '0'; -- impulsion par défaut
            if rx_valide = '1' then
                registre_attente <= rx_data; -- mémoriser
                octet_en_attente <= '1';
            end if;
            if octet_en_attente = '1' and tx_busy = '0' then
                tx_start <= '1'; -- émettre dès que TX libre
                octet_en_attente <= '0';
            end if;
        end if;
    end process;
end architecture;
```

Décision de conception à documenter : ici on mémorise UN octet. Si un 2^e arrive avant que le 1^e soit émis, il est perdu. Pour n'en perdre aucun, il faudrait un **ring buffer** (celui du TD 01 !) entre RX et TX — mentionne cette limite dans ton compte rendu, c'est ce qu'un ingénieur écrit.

Testbench : envoie « VHDL » octet par octet, vérifie l'écho.

2:30-2:50 — (option) Passage sur carte réelle

Si tu as une Basys 3 (ou autre) :

```
# fichier de contraintes .xdc (Basys 3) – extrait
set_property PACKAGE_PIN W5 [get_ports clk]
create_clock -period 10.0 [get_ports clk]          # 100 MHz -> adapter le generic !
set_property PACKAGE_PIN B18 [get_ports rx]        # USB-UART RXD
set_property PACKAGE_PIN A18 [get_ports tx]        # USB-UART TXD
set_property IOSTANDARD LVCMOS33 [get_ports {clk rx tx}]
```

⚠ `generic map (F_CLK => 100_000_000)` pour la Basys 3 ! Synthétise, charge, ouvre un terminal série (`picocom -b 115200 /dev/ttyUSB1`) : ce que tu tapes revient à l'écran → ton matériel parle au PC.

2:50-3:00 — Livrables et barème

Vérifie ta note avec la grille du TP principal (§Livrables). Commit final :

```
git add . && git commit -m "TP2 seance 3 : robustesse + echo hierarchique"
```

Erreurs fréquentes

Symptôme	Cause	Remède
Le glitch génère un octet 0x00	vérif à mi-start absente	ajouter l'état VERIF_START
-2 % échoue déjà	échantillonnage pas au milieu (marge nulle)	revoir le demi-bit initial
Écho perd des octets en rafale	registre d'attente unique (attendu) ou pas de mémorisation	ring buffer si zéro perte requis
Sur carte : rien ne s'affiche	generic F_CLK pas adapté à l'horloge de la carte	100 MHz sur Basys 3
Sur carte : caractères corrompus	mauvais quartz déclaré, ou baudrate terminal ≠	vérifier <code>.xdc</code> et picocom

Bilan du TP 2

Tu as conçu, simulé et validé un périphérique matériel complet avec la méthode pro : spec chiffrée → schéma → code → testbench exhaustif → tests aux limites → intégration. C'est exactement le workflow d'un ingénieur FPGA. Le tout est réutilisable : `uart_tx/rx` te resserviront pour envoyer les mesures d'un projet FPGA vers un PC.

➡ Retour : [TP 2 \(vue d'ensemble\)](#) · [Module 05 — Java](#)

FORMATION EMBARQUÉE

TP3 — Seance 1

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 3 — Fiche de séance 1 : analyse fonctionnelle (2 h)

Aucune ligne de code cette séance. En automatisme, 40 % du travail est l'analyse. Un GRAFCET juste rend le codage trivial ; un GRAFCET bâclé rend le codage impossible. C'est la séance la plus importante du TP.

En-tête pédagogique

Objectifs	Rédiger une liste d'E/S ; tracer un GRAFCET point de vue partie commande ; analyser les défauts avec leurs conditions de réarmement
Prérequis	Module 07 en entier ; TD 07 (auto-maintien, front, GRAFCET)
Outils	Papier/crayon (ou draw.io) ; TIA Portal pas encore nécessaire
Livrable	3 documents : liste d'E/S, GRAFCET, analyse de défauts

Rappel du cahier des charges

Ligne de remplissage de bouteilles : convoyeur → cellule détecte la bouteille sous le bec → arrêt convoyeur, électrovanne ouverte pendant un temps réglable (2-10 s) → 1 s de stabilisation → convoyeur repart. Compteur de lot réglable ; lot atteint → arrêt + message + acquittement. Modes AUTO/MANU. Défauts : bourrage (cellule occupée > 15 s hors remplissage), arrêt d'urgence (NF câblé).

Déroulé minuté

0:00-0:40 — Livrable 1 : la liste d'E/S

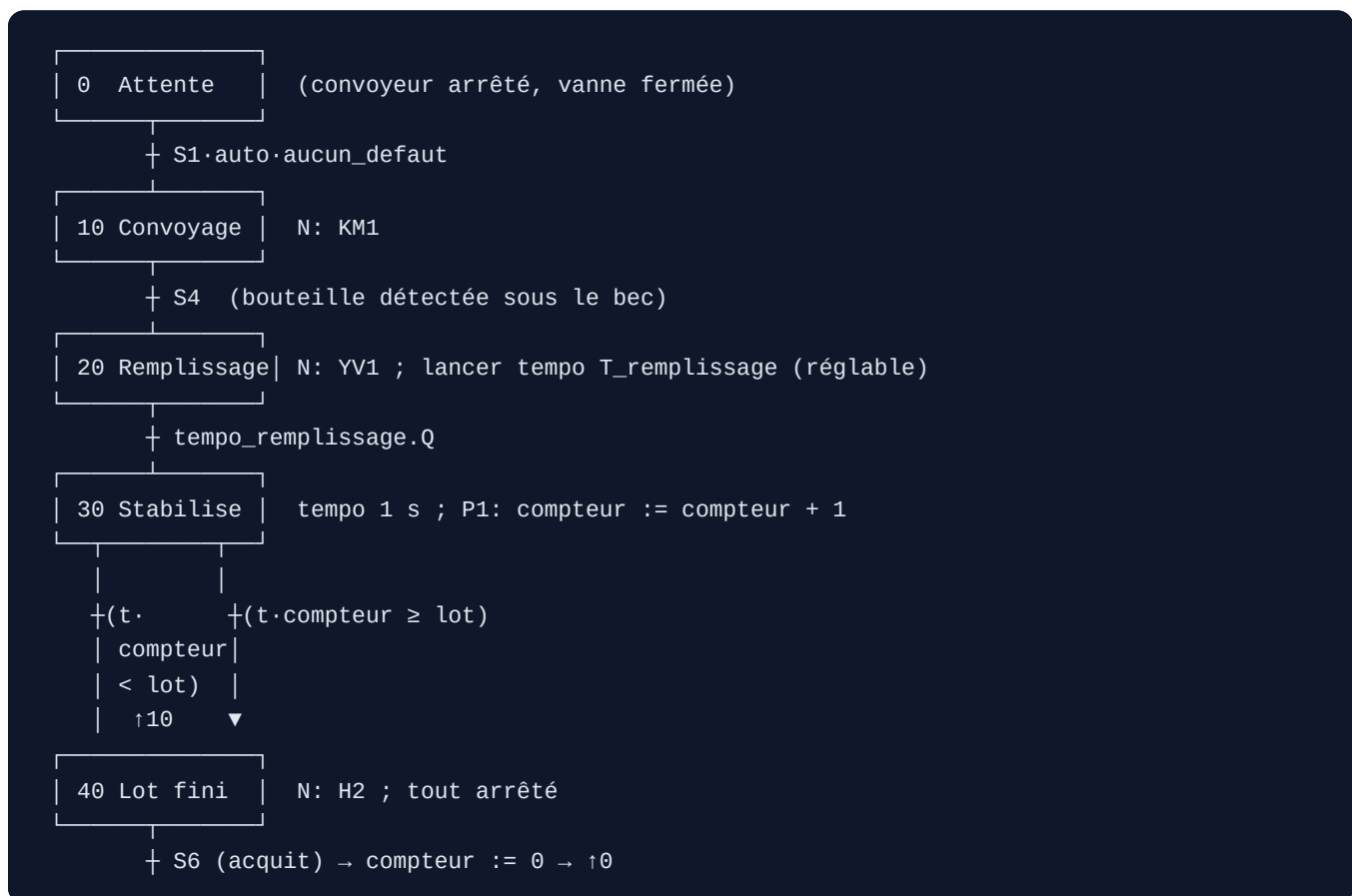
C'est le contrat entre le programme et l'armoire électrique. Complète ce tableau (repère, description, type, adresse, actif) :

Repère	Description	Type	Adresse	Actif
S1	BP Marche	DI	%I0.0	NO
S2	BP Arrêt	DI	%I0.1	NO
S3	Arrêt d'urgence	DI	%I0.2	NF
S4	Cellule bouteille présente	DI	%I0.3	NO
S5	Sélecteur AUTO/MANU	DI	%I0.4	—
S6	BP Acquit défauts	DI	%I0.5	NO
KM1	Moteur convoyeur	DO	%Q0.0	—
YV1	Électrovanne remplissage	DO	%Q0.1	—
H1	Voyant défaut	DO	%Q0.2	—
H2	Voyant lot terminé	DO	%Q0.3	—

Questions à trancher (note tes choix) : - L'arrêt d'urgence : pourquoi NF et pas NO ? (*fil coupé = sécurité déclenchée — sécurité positive*) - Faut-il un capteur « carton/bec en position » ? Pour ce CdC simplifié, non — mais énonce l'hypothèse.

0:40-1:30 — Livrable 2 : le GRAFCET

Trace le GRAFCET point de vue partie commande (les actions = les sorties, les transitions = les capteurs) :



Les 3 règles que le correcteur vérifie : 1. **Chaque transition est un événement observable** par un capteur ou une tempo. « La bouteille est pleine » n'est pas observable ici ; « la tempo est écoulée » l'est. 2. **Une seule étape active à la fois** dans cette séquence linéaire (pas de divergence ET). 3. Les **actions au franchissement** (incrément compteur : qualificatif P1) se distinguent des **actions continues** (N : maintenues tant que l'étape est active).

1:30-2:00 — Livrable 3 : l'analyse des défauts

Un tableau par défaut. C'est ici qu'on gagne ou perd des points en revue :

Défaut	Détection	Action immédiate	Réarmement
Arrêt d'urgence	S3 = 0 (NF)	KM1 et YV1 coupés instantanément, mémorisation défaut	disparition (S3=1) ET S6
Bourrage	S4 = 1 maintenu 15 s hors étapes 20/30	KM1 coupé, défaut mémorisé	disparition cause ET S6
(implicite) mode incohérent	changement AUTO ↔ MANU en cycle	à définir : figer ? revenir à l'étape 0 ?	ton choix argumenté

Le point clé : **le réarmement n'est jamais automatique**. Disparition de la cause + action volontaire de l'opérateur (S6). Une machine qui redémarre seule après un AU est un danger (et hors norme).

✓ **Point de contrôle** : relis ton GRAFCET à voix haute comme si tu l'expliquais à un collègue. Chaque transition doit se dire « quand [capteur] alors [étape suivante] ». Si tu dois dire « quand c'est fini » sans nommer de capteur, la transition est mal posée.

Erreurs fréquentes

Erreur	Pourquoi c'est faux	Correction
Transition « bouteille pleine »	pas de capteur de niveau dans le CdC	c'est la tempo qui décide
Incrément compteur en action N	il s'incrémenterait à chaque cycle automate (ms)	qualificatif P1 (au franchissement)
AU géré comme une transition du GRAFCET	l'AU doit agir depuis N'IMPORTE quel état	logique transversale, hors séquence
Réarmement = juste relâcher l'AU	redémarrage intempestif	exiger S6 en plus

Travail à la maison (30 min)

Ajoute au GRAFCET la gestion du **mode MANU** : soit une branche parallèle, soit (mieux) une logique séparée hors GRAFCET qui court-circuite la séquence. Réfléchis : en MANU, les sécurités (AU, verrouillages) restent-elles actives ? (*réponse : oui, toujours — le mode manuel n'est pas un mode « sans sécurité »*).

➡ Fiche suivante : [Séance 2 — Programme structuré](#)

FORMATION EMBARQUÉE

TP3 — Seance 2

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 3 — Fiche de séance 2 : programme structuré en blocs (3 h)

En-tête pédagogique

Objectifs	Créer un projet TIA + CPU + tags ; structurer en OB/FC/FB/DB ; traduire le GRAFCET en SCL ; centraliser les sorties
Prérequis	Séance 1 : GRAFCET et liste d'E/S validés
Outils	TIA Portal (essai) + PLCSIM
Livrable	Projet qui compile sans warning, table de visualisation prête

Déroulé minuté

0:00-0:20 — Projet, CPU, tags

1. Créer un projet → « Ajouter un appareil » → CPU 1214C DC/DC/DC.
2. **Table des variables API** : saisir toute la liste d'E/S de la séance 1 (noms symboliques !). Ne jamais écrire `%I0.0` dans le code — toujours `"S1_marche"`.
3. Ajouter les mémentos et un DB : `"DB_IHM"` (global, DB optimisé) avec `marche_demandee`, `temps_remplissage_ms` (Int), `taille_lot` (Int), `compteur` (Int), `etape` (Int).

0:20-0:40 — L'architecture imposée

Bloc	Type	Rôle
<code>Main [OB1]</code>	OB	appelle les blocs dans l'ordre, RIEN d'autre
<code>FC_Modes</code>	FC	calcule <code>auto_actif</code> / <code>manu_actif</code> , demande marche (auto-maintien)
<code>FB_Defaults</code>	FB	AU, bourrage (TON 15 s), mémorisation, S6 → <code>aucun_default</code>
<code>FB_Sequence</code>	FB	le GRAFCET en SCL (<code>CASE #etape</code>)
<code>FC_Sorties</code>	FC	seul bloc qui écrit les %Q

Pourquoi ce découpage ? Deux règles d'or de la maintenabilité : 1. **Un seul bloc écrit les sorties.** Quand un technicien cherchera « pourquoi KM1 ne tourne pas », il ouvrira UN réseau. Sorties éparpillées = cauchemar (et double écriture = bug : la dernière gagne). 2. **La sécurité est en aval, en dernier.** Quelle que soit la fantaisie de la séquence, `FC_Sorties` coupe tout sur défaut.

0:40-1:00 — L'ordre d'appel dans OB1

```
// OB1 – orchestration pure
"FC_Modes"();
"FB_Defaults"(DB_Defaults);           // FB → son DB d'instance
"FB_Sequence"(DB_Sequence);
"FC_Sorties"();                       // TOUJOURS en dernier
```

L'ordre compte : les modes et défauts sont calculés AVANT la séquence (qui les lit), et les sorties APRÈS tout le monde.

1:00-2:15 — Coder FB_Sequence en SCL

Interface du FB : Static `etape : Int`, `t_rempli : TON_TIME`, `t_stab : TON_TIME`. Corps :

```
CASE #etape OF
  0: // Attente
    IF "DB_IHM".marche_demandee AND #auto AND #aucun_defaut THEN
      #etape := 10;
    END_IF;

  10: // Convoyage
    IF "S4_bouteille" THEN #etape := 20; END_IF;

  20: // Remplissage
    #t_rempli(IN := TRUE,
              PT := DINT_TO_TIME(INT_TO_DINT("DB_IHM".temps_remplissage_ms)));
    IF #t_rempli.Q THEN
      #t_rempli(IN := FALSE);           // réarmer en quittant l'étape
      #etape := 30;
    END_IF;

  30: // Stabilisation + incrément lot
    #t_stab(IN := TRUE, PT := T#1s);
    IF #t_stab.Q THEN
      #t_stab(IN := FALSE);
      "DB_IHM".compteur := "DB_IHM".compteur + 1; // au franchissement
      IF "DB_IHM".compteur >= "DB_IHM".taille_lot THEN
        #etape := 40;
      ELSE
        #etape := 10;
      END_IF;
    END_IF;

  40: // Lot terminé
    IF "S6_acquit" THEN
      "DB_IHM".compteur := 0;
      #etape := 0;
    END_IF;
END_CASE;
```

Note : les numéros d'étape = ceux du GRAFCET dessiné. Le dessin est le document de référence ; le code n'est que sa traduction. Mets le GRAFCET en commentaire en tête du bloc.

2:15-2:45 — FC_Sorties : la centralisation

```
// SEUL bloc qui écrit les %Q. La sécurité est intégrée ici, en dernier.
"KM1_convoyeur" := ("DB_Sequence".etape = 10 OR "DB_Sequence".etape = 30
                    OR #cmd_manu_convoyeur)
                    AND "FB_Defaults".aucun_defaut;

"YV1_vanne" := ("DB_Sequence".etape = 20 OR #cmd_manu_vanne)
               AND "FB_Defaults".aucun_defaut;

"H1_defaut" := NOT "FB_Defaults".aucun_defaut;
"H2_lot"    := ("DB_Sequence".etape = 40);
```

Observe : même si la séquence bugue et met KM1 à 1 au mauvais moment, `AND aucun_defaut` le coupe. C'est la **défense en profondeur**.

2:45-3:00 — Compilation et table de visu

Compile (Ctrl+B) : **zéro erreur, zéro avertissement** (un warning « variable non initialisée » cache souvent un vrai bug). Crée une table de visualisation `Visu_Cycle` avec : `etape`, toutes les %I, toutes les %Q, `compteur`, les `.Q` des tempos.

✅ **Point de contrôle** : compilation propre + table prête. Commit du projet (export ou dossier archivé + note dans ton journal Git perso).

Erreurs fréquentes

Symptôme	Cause	Remède
Compteur s'incrémente en boucle	incrément dans une action continue, pas au franchissement	mettre l'incrément dans le <code>IF tempo.Q</code>
Tempo qui ne redémarre jamais	<code>IN</code> jamais remis à FALSE	réarmer en quittant l'étape
KM1 reste collé sur défaut	sortie écrite dans plusieurs blocs	centraliser dans FC_Sorties
Warning « accès non initialisé »	variable lue avant écriture	initialiser dans OB100 ou à la déclaration
Séquence bloquée en étape 20	<code>PT</code> mal converti (Int → Time)	<code>DINT_TO_TIME</code> sur des ms

Travail à la maison (45 min)

Code `FB_Defaults` complet : l'AU (mémorisation sur $S3=0$, acquit sur $S6+S3=1$), le bourrage (TON 15 s armé par `S4 AND NOT(etape IN {20,30})`), et la sortie `aucun_defaut`. Teste mentalement : que se passe-t-il si S6 est appuyé alors que l'AU est toujours enfoncé ? (*rien : la cause doit d'abord disparaître*).

➡ Fiche suivante : **Séance 3 — Mise au point PLCSIM**

FORMATION EMBARQUÉE

TP3 — Seance 3

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 3 — Fiche de séance 3 : mise au point sous PLCSIM (2 h)

Règle d'or : la recette de test s'écrit AVANT de tester. Sinon on « bidouille jusqu'à ce que ça marche » sans jamais savoir si tout marche.

En-tête pédagogique

Objectifs	Simuler sans automate ; forcer des entrées ; dérouler une recette de test systématique ; corriger sans régression
Prérequis	Séance 2 : projet compilé, FB_Defaults fait
Outils	TIA Portal + PLCSIM
Livrable	Recette de 7 scénarios déroulée, colonne OK datée

Déroulé minuté

0:00-0:15 — Lancer la simulation

1. « Démarrer la simulation » → PLCSIM s'ouvre, charge le programme.
2. Mettre la CPU en RUN.
3. Ouvrir la table de visualisation **Visu_Cycle** en mode « surveiller » (lunettes). C'est ton tableau de bord.

Dans PLCSIM, tu forces les %I (les capteurs) à la main puisqu'il n'y a pas de vraie machine. Pense à **S3 (AU) = 1** au départ (NF : 1 = OK).

0:15-0:35 — Écrire la recette (livrable 4)

Avant de forcer quoi que ce soit, remplis ce tableau :

#	Scénario	Actions	Résultat attendu	OK ?
1	Cycle nominal	marche, puis S4 pulsé	10 → 20 → 30 → 10, compteur +1	
2	Lot de 3	lot=3, 3 passages	étape 40, H2 allumé	
3	AU en remplissage	S3 → 0 pendant étape 20	YV1 ET KM1 tombent < 1 cycle	
4	Réarmement sans acquit	S3 → 1 seul	rien ne repart	
5	Réarmement complet	S3 → 1 puis S6	cycle reprend	
6	Bourrage	S4 maintenu 16 s hors remplissage	H1, défaut mémorisé	
7	Consigne hors borne	temps=99 s via table	borné à 10 s	

0:35-1:40 — Dérouler la recette

Scénario par scénario, dans PLCSIM :

Scénario 1 — force `marche_demandee` =1 (ou pulse S1) : `etape` passe à 10, KM1 s'allume. Pulse S4 (1 puis 0) : `etape` → 20, YV1 s'allume, la tempo tourne. Après le temps réglé : `etape` → 30, tempo 1 s, puis compteur=1 et retour à 10. **Coche OK ou note l'écart.**

Scénario 3 (le plus important) — pendant l'étape 20 (YV1 allumé), force S3 → 0. Attendu : dans le cycle suivant, YV1 et KM1 retombent, H1 s'allume, la mémoire de défaut est posée. Vérifie sur la table que les deux sorties sont bien à 0. C'est la vérif de sécurité : un défaut qui ne coupe pas les sorties est éliminatoire.

Scénario 7 — depuis la table, écris `temps_remplissage_ms` = 99000 (99 s). Le programme doit le **borner à 10000** (10 s max). Si ton `FC_Modes` / `FB_Sequence` ne borne pas, tu viens de trouver un bug : ajoute le bornage (`IF temps > 10000 THEN temps := 10000`).

1:40-1:55 — Corriger sans régression

Chaque écart trouvé → correction → **re-déroule TOUTE la recette** (pas seulement le scénario qui bloquait). Une correction du scénario 7 peut casser le 1. C'est la discipline « non-régression » : le vrai coût d'un bug est dans les bugs qu'introduit sa correction.

✅ **Point de contrôle** : les 7 scénarios verts, colonne OK datée. Prends une capture d'écran de la table de visu pour le scénario 3 (preuve de la coupure sécurité).

1:55-2:00 — Commit + journal

Note dans ton journal : combien d'itérations recette → correction ? Quel bug t'a le plus surpris ? (souvent : le bornage de consigne ou le réarmement).

Erreurs fréquentes

Symptôme	Cause	Remède
Rien ne démarre	S3 (AU, NF) oublié à 1	forcer S3=1 au départ
S4 déclenche 2 remplissages	pas de front sur S4, ou S4 resté à 1	remettre S4 à 0 après pulse
AU coupe mais tout repart seul au relâché	réarmement automatique	exiger S6
Consigne 99 s appliquée	pas de bornage	ajouter <code>IF > max THEN = max</code>
PLCSIM « déconnecté »	CPU pas en RUN, ou recompilation non rechargée	recharger dans l'appareil

Travail à la maison (30 min)

Ajoute un **8^e scénario** à ta recette : que se passe-t-il si S4 reste collé à 1 (cellule masquée en permanence) ? Le bourrage doit se déclencher 15 s après une bouteille non évacuée. Vérifie-le et documente.

➡ Fiche suivante : **Séance 4 — HMI WinCC**

FORMATION EMBARQUÉE

TP3 — Seance 4

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 3 — Fiche de séance 4 : interface opérateur WinCC (3 h)

En-tête pédagogique

Objectifs	Créer des vues HMI ; lier des objets aux variables API ; boutons à impulsion ; champs bornés par niveau utilisateur ; alarmes
Prérequis	Séance 3 : les 7 scénarios verts
Outils	TIA Portal + WinCC Basic + PLCSIM
Livrable	3 vues, scénarios 2/3/6 rejoués depuis l'écran

Déroulé minuté

0:00-0:20 — Ajouter le pupitre

« Ajouter un appareil » → pupitre KTP700 Basic. La liaison PROFINET avec la CPU se crée dans la vue réseau (relie les deux ports). Les variables HMI se lieront directement aux tags API du projet.

0:20-1:10 — Vue « Conduite »

Objets à poser (bibliothèque WinCC) : - **Boutons Marche/Arrêt** : événement « Appuyer » → mise à 1, « Relâcher » → mise à 0 sur `DB_IHM.marche_demandee` . ⚠ **Bouton à impulsion, jamais bistable**. Rappel du TD 07 : si la liaison HMI tombe pendant qu'un bouton « reste à 1 », l'ordre reste bloqué. La logique de maintien vit dans l'automate (l'auto-maintien de `FC_Modes`), pas dans l'écran. - **Synoptique** : un rectangle « convoyeur » qui change de couleur selon `KM1` , une « bouteille » sous le « bec », le bec animé par `YV1` . - **Texte d'état dynamique** : afficher « Remplissage 3/12 » via un champ qui concatène `compteur` et `taille_lot` . Le mieux : un champ texte dont la visibilité dépend de `etape` (un texte par état). - **Voyant mode** : AUTO/MANU selon S5.

Point de conception : le voyant convoyeur doit refléter le **retour réel** (idéalement un contact auxiliaire du contacteur), pas la recopie de la commande. Un voyant qui montre « ce qu'on a demandé » masque les pannes (contacteur collé/décollé). Dans ce TP simplifié on a `KM1` = commande, mais énonce la limite.

1:10-1:50 — Vue « Réglages »

- Champ d'E/S numérique pour `temps_replissage_ms` : **limites 2000-10000 déclarées** dans les propriétés du champ (l'automate revalide de son côté — défense en profondeur, cf. scénario 7 de la séance 3).
- Champ pour `taille_lot` : limites 1-99.
- **Protection par niveau utilisateur** : crée un utilisateur « Régleur » avec mot de passe ; la vue Réglages exige ce niveau. Un opérateur ne modifie pas les recettes — c'est de la conception, pas du luxe.

1:50-2:30 — Vue « Alarmes »

1. Créer les **alarmes TOR** : dans « Alarmes IHM », une ligne par défaut (AU, bourrage) déclenchée sur le bit correspondant, classe « Errors », texte explicite (« Défaut bourrage — dégager la cellule S4 »).
2. Le lot terminé : classe « Warnings » (info, pas erreur).
3. Poser une **fenêtre des alarmes** sur la vue + un **bouton d'acquiescement**.
4. Texte utile : « Arrêt d'urgence actif — déverrouiller puis acquiescer » vaut mieux que « Défaut 3 ».

2:30-2:55 — Test intégré HMI + PLCSIM

Lance la **simulation du pupitre** (WinCC) en parallèle de PLCSIM. Rejoue **depuis l'écran** (plus depuis la table de forçage) : - Scénario 2 : régler lot=3 dans Réglages, lancer, voir « x/3 » évoluer, H2 et l'alarme « lot terminé » à la fin. - Scénario 3 : déclencher l'AU (force S3=0 dans PLCSIM), voir l'alarme monter, acquiescer après réarmement. - Scénario 6 : provoquer le bourrage, vérifier alarme + voyant.

✓ **Point de contrôle** : les 3 scénarios se pilotent et s'observent entièrement depuis l'IHM, alarmes acquiescables.

2:55-3:00 — Livrables et barème final

Remplis la grille /20 du TP principal ([tp3-siemens-remplissage.md](#)) avec preuves (captures des 3 vues, du déroulé de recette). Commit/archive du projet.

Erreurs fréquentes

Symptôme	Cause	Remède
Bouton marche « collant » (reste actif)	bouton bistable au lieu d'impulsion	événements Appuyer/Relâcher
Consigne acceptée hors borne	limites déclarées seulement côté HMI	borner AUSSI dans l'automate
Alarme jamais affichée	bit de déclenchement mal lié, ou classe muette	vérifier le tag et la fenêtre d'alarmes
Réglages modifiables par tous	pas de niveau utilisateur	protéger la vue par « Régleur »
Synoptique figé	variable HMI non actualisée (cycle d'acquisition)	vérifier la liaison et le cycle

Bilan du TP 3

Tu as mené un mini-projet d'automatisme complet : analyse → E/S → GRAFCET → programme structuré → recette de test → IHM. **Cette méthode est indépendante de la marque** : le TP 4 (Schneider) applique la même démarche sur EcoStruxure. Ce que tu retiens ici (sorties centralisées, sécurité en aval, boutons à impulsion, recette écrite) est valable partout.

➡ Retour : [TP 3 \(vue d'ensemble\)](#) · [Module 08 — Schneider](#)

FORMATION EMBARQUÉE

TP4 — Seance 1

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 4 — Fiche de séance 1 : programme M221 (3 h)

En-tête pédagogique

Objectifs	Créer un projet Machine Expert Basic ; mise à l'échelle analogique ; régulation à hystérésis ; alternance de pompes ; chien de garde capteur
Prérequis	Module 08 ; TD 08 exercices 1-2 ; TP 3 terminé (la méthode n'est pas répétée)
Outils	EcoStruxure Machine Expert – Basic (gratuit) + simulateur M221
Livrable	Programme M221 régulant une cuve, testé au simulateur

Rappel du cahier des charges

Cuve avec mesure de niveau analogique 0-10 V (`%IW0.0` , 0..1000 brut = 0-100 %), deux pompes de remplissage à hystérésis (départ < 40 %, arrêt > 60 %) avec alternance selon l'usure, vanne de soutirage, sécurités (niveau très haut

90 %, très bas < 5 %, capteur figé > 30 s).

Déroulé minuté

0:00-0:20 — Projet et configuration

1. Nouveau projet → TM221CE16R (version Ethernet, pour le Modbus de la séance 2).
2. Onglet configuration : régler l'entrée analogique 0-10 V, adresse IP 192.168.1.50 (le simulateur l'émule).
3. **Table de symboles** complète AVANT le code (réflexe du TP 3) : `NIVEAU_POURMILLE` (%MW10), `CMD_P1` (%Q0.0), `CMD_P2` (%Q0.1), `VANNE` (%Q0.2), `DEF_CAPTEUR` (%M20), `DEMANDE_REPLIR` (%M0).

0:20-0:40 — Mise à l'échelle

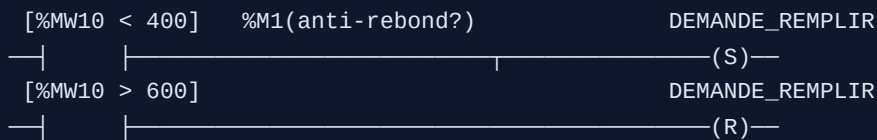
`%IW0.0` (0..1000) → `%MW10` en pour-mille (0..1000 = 0..100,0 %). Sur M221, un bloc opération dans une section « Échelle » :

```
%MW10 := %IW0.0    (ici 1:1 ; si le module rend 0..27648, faire une règle de 3)
```

Convention (TD 08) : entiers ×10, pas de flottants. `470` = 47,0 %.

0:40-1:30 — Régulation à hystérésis (LADDER)

Section « Régulation », avec blocs comparaison :



Départ sous 40 %, arrêt au-dessus de 60 % : la bande morte de 20 points évite que les pompes « battent » autour d'un seuil unique (même principe que l'hystérésis du thermostat Arduino, TD 03). La demande est une **bascule Set/Reset**.

1:30-2:20 — Alternance des pompes

Machine Expert Basic n'a pas de FB utilisateur : implémente l'alternance (TD 08 exercice 2) en LADDER + mots. Logique : - `%MW30` = heures P1, `%MW31` = heures P2 (compteurs d'usure). - Au **front montant** de `DEMANDE_REEMPLIR` : choisir la pompe la moins usée (`%M10` = « choix P1 » si `%MW30 <= %MW31`). - Commandes : `CMD_P1 = DEMANDE_REEMPLIR AND choix_P1 AND NOT def_P1` , symétrique pour P2. - Comptage : un `%TM` cadencé à 1 s (ou le bit système `%S6`) incrémente le compteur de la pompe en marche.

Documente ton choix d'implémentation (LADDER vs liste d'instructions) dans le journal — les deux sont acceptés, l'important est que ce soit lisible.

2:20-2:45 — Chien de garde capteur

Le plus subtil : détecter un capteur figé. Un niveau immobile est **normal** si rien ne bouge — il n'est anormal que si une pompe tourne ou la vanne est ouverte et que le niveau ne varie pas.

```

Condition d'armement : (CMD_P1 OU CMD_P2 OU VANNE) ET |%MW10 - %MW_precedent| < 2
Si maintenue 30 s (%TM à 30 s) → DEF_CAPTEUR
Remise à zéro du %TM dès que le niveau varie de ±2

```

Mémore `%MW10` dans `%MW11` à chaque scrutation pour comparer.

2:45-3:00 — Sécurités (section la plus en aval)

Comme au TP 3 : **une seule section écrit les %Q**, en dernier, avec la sécurité intégrée : - `%MW10 > 900` → `CMD_P1 = CMD_P2 = 0` (interdiction de remplir). - `%MW10 < 50` → `VANNE = 0` (protéger la pompe aval). - `DEF_CAPTEUR` → tout à l'état sûr.

✓ **Point de contrôle** : au simulateur, pilote `%IW0.0` à la souris. Vérifie : hystérésis (départ sous 40, arrêt à 60), alternance visible d'un cycle au suivant (les heures P1/P2 s'équilibrent), niveau haut prioritaire.

Erreurs fréquentes

Symptôme	Cause	Remède
Pompes qui papillonnent	pas d'hystérésis (seuil unique)	bande morte 40/60
Toujours la même pompe	choix non figé au front, ou pas de comptage	choisir au front montant
DEF_CAPTEUR permanent	armement sans condition « qqchose bouge »	armer seulement si pompe/vanne active
Niveau haut ignoré	sécurité pas en aval	section sorties en dernier
%MW10 aberrant	mise à l'échelle du module non faite	vérifier plage brute réelle

Travail à la maison (30 min)

Ajoute la gestion du **report sur défaut** : si la pompe choisie tombe en défaut pendant le remplissage, basculer sur l'autre (si elle est saine). Et l'alarme « aucune pompe disponible » si les deux sont en défaut. C'est l'exercice 2 du TD 08 dans son intégralité.

➡ Fiche suivante : [Séance 2 — Table Modbus](#)

FORMATION EMBARQUÉE

TP4 — Seance 2

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 4 — Fiche de séance 2 : table d'échange Modbus (2 h)

En-tête pédagogique

Objectifs	Concevoir une table d'échange documentée ; distinguer zones lecture/écriture ; implémenter un mot de vie ; borner les entrées réseau
Prérequis	Séance 1 : régulation cuve fonctionnelle
Outils	Machine Expert Basic + simulateur
Livrable	Document de table + section « Échange » dans le programme

Déroulé minuté

0:00-0:40 — Le document de table (livrable clef)

Une table Modbus est un **document contractuel** : Modbus ne transporte que des mots 16 bits, tout le *sens* est dans ce document. Sans lui, le client PC ne peut rien décoder. Rédige-le (reprends le format du TD 08 ex. 3) :

Mot	Adr.	Sens	Contenu	Codage
%MW100	100	API → PC	État : b0=P1, b1=P2, b2=vanne, b3=nivTH, b4=nivTB, b5=defCapteur	bits
%MW101	101	API → PC	Niveau	pour-mille (0-1000)
%MW102	102	API → PC	Heures P1	h ×10
%MW103	103	API → PC	Heures P2	h ×10
%MW104	104	API → PC	Mot de vie	+1/s, boucle 0-65535
%MW110	110	PC → API	Commande : b0=autoriser, b1=acquit	bits
%MW111	111	PC → API	Consigne haute	pour-mille, borné 500-800

Règles à énoncer : - **Zones disjointes** : jamais les deux côtés écrivains du même mot (API → PC et PC → API séparés). Sinon, écrasement mutuel. - **Entiers ×10** plutôt que Real : un flottant occupe 2 mots avec un ordre (word swap) dépendant de l'équipement — source n°1 de bugs Modbus. - **Le mot de vie** : indispensable. Une connexion TCP ouverte ne prouve pas que le programme tourne (la CPU peut être en STOP, figée). Le PC surveille que ce compteur bouge.

0:40-1:20 — Section « Échange » dans le programme

Sur M221, les **%MW** sont nativement lisibles/écrivables en Modbus TCP (holding registers, adresse = numéro du mot). Rien à « publier », mais il faut **recopier** explicitement les états dans la zone API → PC et

lire + borner la zone PC → API :

```
// Recopie des états vers %MW100 (bloc opération, bit à bit)
%MW100 := 0
Si CMD_P1      alors %MW100 := %MW100 OU 1      (b0)
Si CMD_P2      alors %MW100 := %MW100 OU 2      (b1)
Si VANNE       alors %MW100 := %MW100 OU 4      (b2)
Si %MW10>900   alors %MW100 := %MW100 OU 8      (b3)
Si %MW10<50    alors %MW100 := %MW100 OU 16     (b4)
Si DEF_CAPTEUR alors %MW100 := %MW100 OU 32     (b5)

%MW101 := %MW10      // niveau
%MW102 := %MW30 / 360 // secondes → dixièmes d'heure, selon ton codage
```

1:20-1:45 — Mot de vie et bornage

Mot de vie : incrémenter `%MW104` chaque seconde (bit système `%S6` qui bascule à 1 Hz + détection de front + bloc opération). Laisser déborder naturellement (0-65535).

Bornage de la consigne : l'automate ne fait JAMAIS confiance au réseau.

```
Si %MW111 < 500 alors %MW111 := 500
Si %MW111 > 800 alors %MW111 := 800
// puis seulement, utiliser %MW111 comme seuil haut de régulation
```

Un PC (ou un pirate) qui écrit 9999 dans `%MW111` ne doit pas faire déborder la cuve.

✅ **Point de contrôle** : au simulateur, vérifie que `%MW100` reflète bien les états (allume une pompe → b0 passe à 1), que `%MW104` s'incrémente, et qu'écrire 9999 dans `%MW111` le ramène à 800.

1:45-2:00 — Commit + doc

Range le document de table dans le dépôt (`docs/table_modbus.md`) — c'est le livrable que tu donneras au développeur du superviseur (toi-même en séance 3).

Erreurs fréquentes

Symptôme	Cause	Remède
Le PC lit des valeurs incohérentes	pas de recopie état → %MW100	ajouter la section Échange
Consigne 9999 appliquée	pas de bornage côté API	borner à chaque cycle
PC ne détecte pas la panne CPU	mot de vie absent ou figé côté API	%S6 + front + incrément
b0/b1 mélangés	masques de bits erronés	vérifier les puissances de 2
Le PC écrit une %Q directement	zone d'écriture mal conçue	le PC écrit des DEMANDES, l'API valide

Travail à la maison (30 min)

Complète la table avec 3 mots de réserve (%MW105-107, mis à 0) pour les extensions futures — un réflexe pro : on prévoit toujours de la place. Et documente : que doit faire le PC s'il lit %MW104 identique 3 fois de suite ? (*afficher une alarme « automate figé » — implémenté séance 3*).

➡ Fiche suivante : [Séance 3 — Client de supervision PC](#)

FORMATION EMBARQUÉE

TP4 — Seance 3

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 4 — Fiche de séance 3 : client de supervision PC (3 h)

En-tête pédagogique

Objectifs	Lire/écrire un automate en Modbus TCP depuis un PC ; surveiller un mot de vie ; journaliser ; faire des essais croisés automate ↔ PC
Prérequis	Séance 2 : table Modbus implémentée et testée
Outils	Machine Expert Basic + simulateur ; Python + <code>pip install pymodbus</code>
Livrable	<code>supervision.py</code> fonctionnel + 5 essais croisés documentés

Déroulé minuté

0:00-0:20 — Vérifier le lien avec un client générique d'abord

Avant d'écrire du code, valide le bus avec un outil tout fait (méthode : une inconnue à la fois). QModMaster ou `mbpoll` :

```
mbpoll -m tcp -a 1 -r 100 -c 5 -t 4 192.168.1.50 # lit %MW100..104
```

Tu dois voir les 5 mots. Si le simulateur n'expose pas le port 502, active le serveur Modbus dans sa config. **Ce test isole** : si `mbpoll` marche mais pas ton script, le bug est dans ton script ; sinon, dans l'automate.

0:20-1:30 — Écrire `supervision.py`

Pars du corrigé complet fourni : `code/python/supervision.py` . Il fait déjà tout (lecture, affichage tableau de bord, mot de vie, CSV, commandes clavier en thread). Ton travail : le **comprendre ligne par ligne** et l'adapter à ta table. Points à saisir :

- La lecture groupée `read_holding_registers(address=100, count=5)` : un seul échange réseau pour 5 mots cohérents (mieux que 5 échanges).
- Le **décodage des bits** d'état avec des masques (`etat & 0x01` pour P1) — exactement le C du module 01, en Python.
- La **surveillance du mot de vie** : compteur figé 3 fois → alarme.
- Le thread clavier : les commandes ne bloquent jamais la scrutation.
- L'écriture `write_register(111, valeur)` : le PC envoie une **demande**, l'automate borne et valide (jamais l'inverse).

Lance-le : `python3 supervision.py 192.168.1.50` .

1:30-2:30 — Les 5 essais croisés (le cœur du TP)

Déroule et **documente chacun** (capture + observation) :

#	Scénario	Manip	Attendu
1	Cycle normal	laisser tourner	niveau oscille 40 ↔ 60, heures P1/P2 s'équilibrent
2	Consigne	taper <code>consigne 650</code>	seuil haut passe à 65 % ; <code>consigne 950</code> → borné à 80 %
3	Panne CPU	mettre le simulateur en STOP	le PC détecte le mot de vie figé < 3 s, alarme affichée
4	Commande stop	taper <code>stop</code> pendant un remplissage	pompes coupées PAR L'AUTOMATE (le PC n'a écrit qu'une demande)
5	Capteur figé	bloquer <code>%IW0.0</code> pendant qu'une pompe tourne	DEF_CAPTEUR après 30 s, visible côté PC, positions sûres

L'essai 3 est le plus formateur : c'est lui qui distingue une vraie supervision d'une démo. Un lien TCP « connecté » ne prouve pas que l'automate vit. Le mot de vie est la seule garantie.

✓ **Point de contrôle** : les 5 essais documentés, essai 3 en évidence.

2:30-2:55 — (option) Version Java

Remplace le client Python par ton programme Java du module 05 §7.1 (avec la bibliothèque j2mod) : **même table, même comportement**. Constat à écrire : la table d'échange documentée rend le langage du client indifférent — c'est exactement son rôle. Python pour prototyper, Java pour un superviseur d'entreprise, le contrat Modbus ne change pas.

2:55-3:00 — Barème final

Remplis la grille /20 du TP principal ([tp4-schneider-cuve.md](https://github.com/tp4-schneider-cuve)) avec preuves. Commit du script + journal CSV d'exemple + document de table.

Erreurs fréquentes

Symptôme	Cause	Remède
<code>Connection refused</code>	serveur Modbus non activé sur le simulateur	activer dans la config M221
pymodbus : <code>.registers</code> vide	<code>.isError()</code> non testé, adresse hors zone	tester l'erreur, vérifier l'adresse
Le PC croit tout OK alors que la CPU est STOP	mot de vie non surveillé	comparer à la valeur précédente
<code>stop</code> ne coupe pas les pompes	le PC écrit une %Q au lieu d'une demande	le PC écrit %MW110, l'API décide
Valeurs ×10 mal affichées	oubli de diviser par 10 à l'affichage	<code>niveau/10</code>

Bilan du TP 4

Tu as construit une chaîne **automate** ↔ **supervision** complète et vu les deux écosystèmes (TIA au TP 3, EcoStruxure ici) sur la même méthode. Le profil « automaticien qui parle aussi informatique » (Modbus, Python/Java, tables d'échange) est très recherché. Range ce projet dans ton portfolio à côté du TP 3 : ensemble, ils montrent que tu maîtrises la démarche, pas juste un logiciel.

 Retour : [TP 4 \(vue d'ensemble\)](#) · [Module 10 — STM32](#)

FORMATION EMBARQUÉE

TP5 — Seance 1

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 5 — Fiche de séance 1 : socle STM32 (horloges, UART, GPIO) (2 h)

En-tête pédagogique

Objectifs	Créer un projet CubeIDE ; comprendre le clock tree ; rediriger printf ; heartbeat non bloquant ; bouton par interruption
Prérequis	Module 10 + TD 10 faits ; TP 1 (l'esprit non bloquant)
Outils	STM32CubeIDE ; Nucleo-F411RE (adapter si autre carte)
Livrable	Bannière série + heartbeat + bouton pris en compte

Déroulé minuté

0:00-0:20 — Projet et clock tree

1. **File → New → STM32 Project** → Board Selector → Nucleo-F411RE → initialiser les périphériques par défaut : **oui**.
2. Onglet **Clock Configuration** : vise **100 MHz** sur HCLK (tape 100 dans la case, CubeMX résout la PLL). Note dans ton journal les valeurs M/N/P choisies, et **vérifie que APB1 ≤ 50 MHz** (au-delà, certains périphériques ne suivent pas). Comprendre cet arbre, c'est comprendre d'où vient la fréquence de chaque timer/bus (§3 du module 10).

0:20-0:50 — Redirection de printf

Génère le code (Alt+K). Dans **main.c**, entre les balises USER CODE (rappel : tout code hors de ces balises est **écrasé** à la régénération) :

```
/* USER CODE BEGIN 0 */
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, 10);
    return ch;
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
printf("=== Station meteo STM32 ===\r\n");
printf("Build : %s %s\r\n", __DATE__, __TIME__); // savoir QUEL firmware tourne
/* USER CODE END 2 */
```

Ouvre un terminal série (115200) sur le port ST-Link virtuel (**/dev/ttyACM0** ou COMx).  **Point de contrôle 1** : la bannière s'affiche au reset.

Astuce : la bannière avec **__DATE__ / __TIME__** est un réflexe pro — en debug, elle te dit si la carte

exécute bien ta dernière compilation.

0:50-1:20 — Heartbeat non bloquant

La LED LD2 clignote à 1 Hz sans `HAL_Delay` (le socle doit déjà être non bloquant, comme le TP 1) :

```
/* USER CODE BEGIN WHILE */
uint32_t t_led = 0;
while (1) {
    uint32_t now = HAL_GetTick();           // ms depuis le démarrage
    if (now - t_led >= 500) {
        t_led = now;
        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    }
}
/* USER CODE END WHILE */
}
```

C'est le `millis()` d'Arduino, version STM32. Même motif de soustraction robuste au débordement.

1:20-1:50 — Bouton par interruption EXTI

Dans CubeMX : la broche du bouton bleu (PC13, `B1`) est déjà en `GPIO_EXTI`. Vérifie que l'interruption `EXTI15_10` est activée dans l'onglet NVIC. Régénère, puis :

```
/* USER CODE BEGIN PV */
volatile uint8_t verbeux = 1;           // volatile : modifié en ISR
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == B1_Pin) {
        verbeux = !verbeux;           // ISR courte : un drapeau, point
    }
}
/* USER CODE END 0 */
```

Utilise `verbeux` dans la boucle pour activer/couper des logs. Les règles d'ISR du module 00 s'appliquent : courte, `volatile` sur la variable partagée.

✓ **Point de contrôle 2** : appui bouton → la verbosité change (logs qui apparaissent/disparaissent), heartbeat toujours régulier.

1:50-2:00 — Structure du dépôt + commit

Crée `Core/Src/drivers/` (les drivers des séances suivantes y iront). Init Git sur le projet, `.gitignore` pour `Debug/` et les artefacts. Commit : `git commit -m "TP5 seance 1 : socle horloges + printf + heartbeat + bouton"`.

Journal : note les valeurs M/N/P de ta PLL et la fréquence de chaque bus (APB1, APB2). Tu en auras besoin pour calculer les prescalers des timers (séance 3).

Erreurs fréquentes

Symptôme	Cause	Remède
Rien au terminal	mauvais port, vitesse $\neq$ 115200, printf non redirigé	vérifier <code>__io_putchar</code> et le port ST-Link
Code disparu après régénération CubeMX	code hors des balises USER CODE	toujours entre BEGIN/END
Bouton sans effet	EXTI non activé dans NVIC, ou mauvais pin	vérifier onglet NVIC
Heartbeat irrégulier	un HAL_Delay traîne dans la boucle	interdit : utiliser HAL_GetTick
Warning float dans printf	<code>%f</code> sans l'option « use float with printf »	cocher l'option projet, ou éviter %f

Travail à la maison (30 min)

Ajoute un **anti-rebond logiciel** au bouton : le callback EXTI peut être appelé plusieurs fois par appui (rebonds). Enregistre `HAL_GetTick()` dans l'ISR et ignore les appuis à moins de 200 ms d'écart. Constate la différence.

➡ Fiche suivante : [Séance 2 — Driver BME280 maison](#)

FORMATION EMBARQUÉE

TP5 — Seance 2

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 5 — Fiche de séance 2 : driver BME280 écrit maison (3 h)

En-tête pédagogique

Objectifs	Lire un datasheet ; écrire un driver I2C sans bibliothèque ; arithmétique en point fixe ; gérer les erreurs bus sans bloquer
Prérequis	Séance 1 ; module 10 §4.5 (I2C HAL) ; module 01 §2.1 (point fixe)
Outils	CubeIDE ; capteur BME280 (ou potentiomètre en substitut) ; datasheet Bosch BME280 ouvert
Livrable	<code>drivers/bme280.{c,h}</code> retournant T/P/H plausibles, erreurs gérées

Interdiction d'utiliser une bibliothèque toute faite. Le but EST d'écrire le driver. C'est ce qui te sépare d'un « assembleur de bibliothèques » et fait de toi un développeur firmware.

Déroulé minuté

0:00-0:30 — Configurer I2C et scanner le bus

1. CubeMX : activer I2C1 (PB8/PB9 sur Nucleo), 100 kHz (Standard Mode). Régénérer.
2. **Scanner I2C** (le « hello world » du bus) :

```
for (uint8_t a = 1; a < 128; a++)
    if (HAL_I2C_IsDeviceReady(&hi2c1, a << 1, 1, 5) == HAL_OK)
        printf("trouve : 0x%02X\r\n", a);
```

Attendu : `0x76` ou `0x77` . ⚠ **Piège n°1** : la HAL attend l'adresse **décalée d'un bit à gauche** (`a << 1`), contrairement à Wire d'Arduino.

1. Lire le registre `id` (0xD0) : doit répondre **0x60** (l'identifiant du BME280). C'est ta première transaction réussie.

0:30-1:00 — L'interface du driver

`drivers/bme280.h` — l'interface imposée (pense « ce que l'appelant a le droit de savoir ») :

```

#ifndef BME280_H
#define BME280_H
#include "stm32f4xx_hal.h"
#include <stdbool.h>
#include <stdint.h>

typedef struct {
    I2C_HandleTypeDef *i2c;
    uint8_t adresse;           // déjà décalée <<1
    uint16_t dig_T1; int16_t dig_T2, dig_T3;
    uint16_t dig_P1; int16_t dig_P2, dig_P3, dig_P4, dig_P5,
                                dig_P6, dig_P7, dig_P8, dig_P9;
    uint8_t dig_H1, dig_H3; int16_t dig_H2, dig_H4, dig_H5; int8_t dig_H6;
    int32_t t_fine;           // intermédiaire de compensation
} bme280_t;

bool bme280_init(bme280_t *dev, I2C_HandleTypeDef *i2c, uint8_t addr7);
bool bme280_lire(bme280_t *dev, int32_t *temp_centiemes,
                uint32_t *press_pa, uint32_t *hum_milliemes);

#endif

```

Sorties en **point fixe** (centièmes de °C, Pa, millièmes de %) — **aucun float** dans le driver (module 01 : sur un petit micro on évite le flottant, et de toute façon les formules Bosch sont en entiers).

1:00-2:15 — Implémenter (datasheet à côté)

Étapes guidées :

1. **bme280_init** :
2. vérifier l'id (0xD0 → 0x60), sinon **return false** ;
3. lire les registres d'**étalonnage** (0x88..0xA1 pour T/P, 0xE1..0xE7 pour H) — le capteur stocke ses coefficients de correction en usine ;
4. configurer : **ctrl_hum** (0xF2), **ctrl_meas** (0xF4, oversampling ×1 + mode normal), **config** (0xF5).
5. **bme280_lire** :
6. lire **en rafale** les 8 octets 0xF7..0xFE (**HAL_I2C_Mem_Read** avec longueur 8) — une seule transaction pour des valeurs cohérentes entre elles ;
7. recomposer les mesures brutes 20 bits ;
8. appliquer les **formules de compensation** de la datasheet §4.2.3 (recopier ces formules est normal ; les comprendre est l'objectif).
9. **Gestion d'erreur** : chaque **HAL_I2C_...** qui échoue → **return false** . Le driver ne fait **jamais** de **printf** et ne bloque jamais plus que ses timeouts. C'est l'appelant qui décide quoi faire d'un échec.

Corrigé de référence de l'esprit (structure, point fixe, gestion d'erreur) : le driver BME280 est long ; le TP principal §Séance 2 en donne l'ossature. Si tu n'as pas le capteur : simule **bme280_lire** en renvoyant des valeurs dérivées d'un potentiomètre (l'architecture est le vrai objectif).

2:15-2:45 — Tester dans main

```
bme280_t capteur;
if (!bme280_init(&capteur, &hi2c1, 0x76))
    printf("BME280 absent !\r\n");

// dans la boucle (toutes les 1 s, non bloquant) :
int32_t t; uint32_t p, h;
if (bme280_lire(&capteur, &t, &p, &h))
    printf("T=%ld.%02ld C P=%lu Pa H=%lu.%03lu %%\r\n",
          t/100, t%100, p, h/1000, h%1000);
else
    printf("lecture BME280 echouee\r\n");
```

✓ **Point de contrôle** : valeurs plausibles ($T \approx 20-25$ °C, $P \approx 101325$ Pa, $H \approx 40-60$ %). Puis **débranche SDA en cours de route** : les `printf` doivent afficher « lecture echouee » sans que le programme gèle. Un driver qui gèle sur une déconnexion I2C est inutilisable en production.

2:45-3:00 — Commit + journal

`git commit -m "TP5 seance 2 : driver BME280 maison, point fixe, erreurs gerees"`. Journal : quelle formule de compensation t'a le plus surpris ? As-tu vérifié tes types (les coefficients signés/non signés du datasheet sont un piège) ?

Erreurs fréquentes

Symptôme	Cause	Remède
Scanner ne trouve rien	adresse non décalée, SDA/SCL inversés, pull-ups absentes	<code>a<<1</code> , vérifier câblage
id ≠ 0x60	mauvaise adresse, ou c'est un BMP280 (id 0x58)	vérifier la référence du module
Température ×256 ou absurde	coefficients d'étalonnage mal lus (signé/non signé)	respecter les types du datasheet
Valeurs figées	lecture pas en rafale (registres incohérents)	lire les 8 octets d'un coup
Gèle sur déconnexion	pas de test du retour HAL	<code>return false</code> sur échec, timeout court

Travail à la maison (45 min)

Écris un **test du driver compilable sur PC** : remplace les appels HAL par un « mock » qui renvoie des octets fixes (un tableau simulant les registres du capteur), et vérifie que ta compensation produit les valeurs attendues. C'est ainsi qu'on teste un driver sans le matériel (mentionné dans le sujet d'examen A « excellence »).

➡ Fiche suivante : [Séance 3 — ADC + DMA et console](#)

FORMATION EMBARQUÉE

TP5 — Seance 3

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 5 — Fiche de séance 3 : ADC + DMA circulaire et console (2 h)

En-tête pédagogique

Objectifs	Configurer un ADC multi-canaux en DMA circulaire ; comprendre le transfert sans CPU ; réutiliser le ring buffer pour une console UART
Prérequis	Séances 1-2 ; module 10 §4.4/§4.6 ; TD 01 (ring buffer)
Outils	CubeIDE ; potentiomètre sur PA0
Livrable	Tableau ADC rempli en tâche de fond + console à commandes

Déroulé minuté

0:00-0:40 — ADC + DMA circulaire

Le motif star du STM32 : l'ADC scanne plusieurs canaux et **remplit un tableau tout seul**, sans le CPU, en boucle. Configuration CubeMX : 1. ADC1 : activer 2 canaux (IN0 = PA0 potentiomètre, IN16 = capteur de température interne), **Scan Conversion Mode = Enabled**, **Continuous Conversion = Enabled**. 2. DMA : ajouter une requête ADC1, **mode Circular**, direction Peripheral → Memory, données Half Word (16 bits). 3. Régénérer.

```
/* USER CODE BEGIN PV */
volatile uint16_t adc[2];           // rempli par le DMA en tâche de fond
/* USER CODE END PV */

/* USER CODE BEGIN 2 */
HAL_ADC_Start_DMA(&hadc1, (uint32_t *)adc, 2); // et... c'est tout !
/* USER CODE END 2 */
```

Après ce `Start_DMA`, `adc[0]` et `adc[1]` se mettent à jour en permanence sans une seule ligne dans la boucle. C'est le DMA (Direct Memory Access) qui copie périphérique → mémoire.

0:40-1:00 — Moyenne glissante et la démonstration DMA

Ajoute une moyenne glissante sur 16 échantillons de `adc[0]` (réutilise l'esprit du TD 01 / la moyenne glissante du TD 07 §2). Affiche la consigne convertie toutes les 500 ms.

✓ **Point de contrôle DMA (le plus formateur)** : pose un **point d'arrêt** dans la boucle, ouvre la vue « Live Expressions » ou la vue mémoire sur `adc`. Tourne le potentiomètre pendant que le CPU est **arrêté** au point d'arrêt : `adc[0]` **continue de changer**. Preuve visuelle que le transfert se fait sans le CPU. Savoir montrer ça = avoir compris le DMA.

1:00-1:50 — Console UART à ring buffer

Réutilise le ring buffer du TD 01 (`code/c/ring_buffer.c`) et le pattern console du TD 10 exercice 2.
Réception par interruption :

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART2) {
        rb_put(&rb_rx, octet_rx);
        HAL_UART_Receive_IT(&huart2, &octet_rx, 1); // RÉARMER (piège n°1)
    }
}
```

Commandes à implémenter : `status` (mesures + uptime + version), `log on|off`, `seuil <valeur>`. L'ISR ne fait que stocker ; la boucle vide le ring buffer, assemble les lignes (bornées !) et les traite. **Règle « ISR courte »** du module 00 en pratique.

✓ **Point de contrôle console** : `status` renvoie les mesures ADC + BME280 + uptime ; colle 500 caractères d'un coup dans le terminal → pas de crash, pas d'octets mélangés (le ring buffer absorbe).

1:50-2:00 — Commit + journal

`git commit -m "TP5 seance 3 : ADC+DMA circulaire + console ring buffer"`. Journal : décris ce que tu as observé au point d'arrêt (le tableau ADC qui bouge CPU arrêté). C'est LA preuve que le DMA travaille en parallèle.

Erreurs fréquentes

Symptôme	Cause	Remède
<code>adc[]</code> figé	DMA pas en Circular, ou Start_DMA absent	mode Circular + longueur = nb canaux
<code>adc[0]</code> et <code>adc[1]</code> inversés	ordre de rang des canaux dans CubeMX	vérifier l'ordre de scan
Console reçoit 1 seul caractère	<code>Receive_IT</code> pas réarmé dans le callback	réarmer à chaque octet
Octets mélangés en collant du texte	pas de ring buffer, parsing dans l'ISR	ISR = stocker, boucle = parser
Débordement de ligne	tampon de ligne non borné	<code>if (pos < taille-1)</code>

Travail à la maison (30 min)

Compare le capteur de température **interne** du STM32 (`adc[1]` , à convertir avec la formule du reference manual) et le BME280. Ils diffèrent (le capteur interne mesure la puce, qui chauffe). Écris dans ton journal l'écart observé et pourquoi — c'est un piège classique quand on croit mesurer l'ambiance avec le capteur interne.

➡ Fiche suivante : **Séance 4 — Architecture FreeRTOS**

FORMATION EMBARQUÉE

TP5 — Seance 4

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 5 — Fiche de séance 4 : architecture FreeRTOS (3 h)

En-tête pédagogique

Objectifs	Structurer un firmware en tâches ; communiquer par queues ; ISR compatibles RTOS ; surveiller les piles ; valider par essais
Prérequis	Séances 1-3 ; module 06 §5 (FreeRTOS) ; module 10 §7
Outils	CubeIDE (FreeRTOS CMSIS-V2) ; le montage complet
Livrable	4 tâches communicantes + 4 essais de validation documentés

Déroulé minuté

0:00-0:25 — Activer FreeRTOS

- CubeMX : `Middleware` → `FREERTOS` → interface `CMSIS_V2`.
- Point critique** : quand CubeMX le demande, déplacer la base de temps HAL de SysTick vers un timer dédié (ex. TIM10). Pourquoi ? SysTick appartient désormais à l'ordonnanceur RTOS ; la HAL a besoin de sa propre base pour `HAL_Delay` / timeouts. L'oublier = comportements erratiques.
- Régler `configCHECK_FOR_STACK_OVERFLOW = 2` (détection pendant la mise au point).

0:25-0:45 — Déclarer les tâches et queues

Architecture imposée :

Tâche	Priorité	Période	Rôle
<code>T_Capteur</code>	normale	1 s	lit le BME280 → pousse une <code>mesure_t</code> dans <code>q_mesures</code>
<code>T_Traitement</code>	normale	sur message	moyennes, seuil → pousse vers <code>q_affichage</code>
<code>T_Console</code>	basse	sur caractère	console série (bloquée sur queue RX)
<code>T_Heartbeat</code>	la plus basse	500 ms	LED vie + surveillance des piles

Créer les tâches et deux `osMessageQueue` dans CubeMX (ou en code).

0:45-1:45 — Coder les tâches

Règle d'or : **toute communication par queue**, aucune variable globale partagée.

```

void T_Capteur(void *arg) {
    for (;;) {
        mesure_t m;
        if (bme280_lire(&capteur, &m.t, &m.p, &m.h))
            osMessageQueuePut(q_mesures, &m, 0, 0);
        osDelay(1000);           // rend le CPU pendant 1 s
    }
}

void T_Traitement(void *arg) {
    mesure_t m;
    for (;;) {
        if (osMessageQueueGet(q_mesures, &m, NULL, osWaitForever) == osOK) {
            // moyennes, comparaison seuil...
            osMessageQueuePut(q_affichage, &m, 0, 0);
        }
    }
}

```

Points clés : - `osDelay` (bloquant-coopératif) et non une attente active : les tâches de priorité inférieure tournent pendant ce temps. - **L'ISR UART** ne fait que pousser l'octet dans une queue avec la variante `...FromISR`. Et sa **priorité NVIC** doit être numériquement $\geq$ `configMAX_SYSCALL_INTERRUPT_PRIORITY` (module 10 §4.7) — sinon l'appel RTOS depuis l'ISR plante. C'est l'erreur de mise en route n°1. - Si tu partages l'I2C entre deux tâches, il faut un **mutex** ; ici une seule tâche (`T_Capteur`) y touche → pas besoin. **Explique ce choix** dans ton compte rendu (montrer qu'on sait quand un mutex est nécessaire vaut autant que d'en mettre un).

1:45-2:00 — Surveillance des piles

Dans `T_Heartbeat`, log périodiquement la marge de pile de chaque tâche :

```

printf("pile capteur=%lu traitement=%lu\r\n",
       osThreadGetStackSize(h_capteur), osThreadGetStackSize(h_traitement));

```

Et implémente le hook `vApplicationStackOverflowHook` : LED rouge fixe + `printf` du nom de la tâche fautive. Une pile trop petite est LE bug RTOS sournois (corruption aléatoire) — cette surveillance le rend visible.

2:00-2:50 — Les 4 essais de validation

#	Essai	Manip	Attendu
1	Endurance	laisser tourner 10 min	mesures stables, marges de pile stables (pas de dérive)
2	Capteur débranché en marche	retirer SDA	T_Capteur signale, les autres tâches vivent, reprise au rebranchement
3	Spam console	coller 1 Ko de texte	pas de crash, pas d'octets mélangés
4	Point d'arrêt dans T_Traitement 5 s puis reprise	debugger	le système rattrape : messages en attente traités

Essai 4 — le plus formateur : pendant la pause, `T_Capteur` continue de produire des mesures. Où vont-elles ? Dans `q_mesures`. Si la queue déborde (petite + pause longue), que se passe-t-il ?

`osMessageQueuePut` avec timeout 0 les **refuse** (retour $\neq$ `osOK`). **Documente ce comportement** : il n'y a pas une « bonne » réponse unique, mais il faut savoir laquelle tu as choisie (perdre les vieilles ? bloquer ? agrandir la queue ?) et pourquoi.

✅ **Point de contrôle** : les 4 essais documentés, essai 4 analysé.

2:50-3:00 — Barème + README de projet

Remplis la grille /20 du TP principal ([tp5-stm32-freertos.md](#)). Rédige le README du projet (schéma de câblage, photo, mode d'emploi console) — **c'est le projet à mettre en tête de ton portfolio firmware**. Commit final.

Erreurs fréquentes

Symptôme	Cause	Remède
Plante au démarrage RTOS	base de temps HAL restée sur SysTick	la mettre sur TIM10
<code>configASSERT</code> dans l'ISR UART	priorité NVIC trop haute (numériquement trop basse)	priorité $\geq$ <code>configMAX_SYSCALL...</code>
Corruption aléatoire de données	pile de tâche trop petite	augmenter, surveiller le high water mark
Une tâche affame les autres	attente active au lieu d' <code>osDelay</code>	<code>osDelay</code> / attente bloquante sur queue
Messages perdus en rafale	queue trop petite	dimensionner + choisir une politique

Bilan du TP 5 et de la formation

Ton dépôt contient maintenant : un driver I2C bare-metal écrit à la main, du DMA, une architecture RTOS propre avec queues et surveillance de pile, et des essais documentés. **C'est exactement ce qu'un**

recruteur firmware ouvre en premier. Avec les TP 1-4, tu couvres Arduino/IoT, FPGA, et les deux mondes automate — un profil « systèmes embarqués » complet.

Prochaine étape : un des [projets d'évaluation finale](#) (30-40 h) pour consolider, puis publie tout sur GitHub avec les barèmes remplis. Bon courage !

 Retour : [TP 5 \(vue d'ensemble\)](#) · [Parcours & ressources](#)